

Sicurezza - Soluzione Esercizi (25-26)

Oualid Maftah

2026

Contents

1	UDP base e ordine avvio	3
2	UDP porte specifiche	4
3	UDP porte privilegiate	5
4	UDP localhost e hostname	6
5	UDP macchine diverse	7
6	UDP riutilizzo porta	8
7	UDP limite richieste	9
8	UDP invio numeri	10
9	UDP sommatrice	11
10	UDP sommatrice inaffidabilita	12
11	UDP sommatrice multiclient	14
12	TCP base	16
13	TCP sommatrice	17
14	TCP sommatrice bloccante	18
15	TCP sommatrice affidabilita	19
16	TCP trasferimento file	21
17	UDP calcolo remoto	22
18	TCP calcolo remoto	24
19	TCP servizio telepass	26
20	TCP terminale remoto	28
21	Web polling	29
22	Web server	30

23 Web form GET	32
24 Web form POST	34
25 Web server URL	36
26 Upload file server web	38
27 Web server CGI	40
28 Web Socket Chat	42
29 Webservice	44
30 Client Java	46
31 Numeri Primi	48
32 Thread REST	50
33 WebSocket Wireshark	52
34 WebSocket Restyling	54
35 WebSocket porta 80	56
36 WebSocket rinomina eventi	57
37 WebSocket test dispositivi	58
38 WebSocket typing	59
39 MQTT retain	60
40 MQTT due sensori	61
41 MQTT temp e umidita	63
42 MQTT wireshark	64
43 MQTT messaggistica univr	65

1 UDP base e ordine avvio

DOMANDA

Lanciare prima il server e poi il client. Cosa si osserva? Invertire la sequenza di lancio. Cosa si osserva?

RISPOSTA

Ecco cosa si osserva nei due diversi casi, essendo la comunicazione basata su UDP, che è un protocollo connectionless (senza connessione):

- **Lanciando prima il server e poi il client:** La comunicazione avviene con successo. Il server viene avviato e si mette in attesa (`UDPReceive`). Quando viene avviato il client, questo invia il suo messaggio al server e si mette in attesa della risposta. Il server riceve il pacchetto, stampa il contenuto a schermo e invia la risposta. Il client riceve la risposta dal server, la stampa e termina la sua esecuzione. Il server (grazie al ciclo `while(true)`) torna in attesa del prossimo client.
- **Invertendo la sequenza di lancio (prima il client e poi il server):** Si verifica un blocco (entrambi rimangono in attesa all'infinito). Quando il client viene avviato per primo, invia subito il pacchetto UDP verso la porta 35000 e si mette in attesa (`UDPReceive`) della risposta. Poiché il server non è ancora attivo, non c'è nessuna applicazione in ascolto su quella porta. Essendo UDP senza connessione e senza garanzia di consegna, l'invio va a buon fine dal punto di vista del client, ma il pacchetto viene scartato dal sistema operativo di destinazione (e perso). Quando successivamente viene avviato il server, questo si mette in attesa di nuovi messaggi. A questo punto entrambi i programmi si trovano bloccati indefinitamente nelle rispettive `UDPReceive`: il client aspetta una risposta a un messaggio andato perso, mentre il server aspetta un nuovo messaggio che il client non invierà mai più.

2 UDP porte specifiche

DOMANDA

2) Modificare i sorgenti per mettere il server che riceve sulla porta 10000 e il client che trasmette dalla propria porta 30000 (ogni modifica dei sorgenti richiede una loro ricompilazione)

RISPOSTA

- Modificato `serverUDP.c` per metterlo in ascolto sulla porta 10000 (`#define SERVERPORT 10000`).
- Modificato `clientUDP.c` per fare in modo che la sua socket venga creata sulla porta 30000 (`createUDPInterface(30000)`).
- Modificato `clientUDP.c` affinché la `UDPSend` invii il pacchetto alla porta di destinazione 10000.

3 UDP porte privilegiate

DOMANDA

3) Mettere il server in ascolto sulla porta 100 e osservare cosa succede

- Bisogna modificare anche il client? Dove?
- Per chi usa il proprio PC con Linux o una virtual machine Linux, lanciare il server con il comando “`sudo ./serverUDP`” e osservare cosa cambia

RISPOSTA

Ecco le risposte alle tue domande:

- **Bisogna modificare anche il client? Dove?**

Sì, è indispensabile modificare anche il client. Affinché il client riesca a contattare il server, deve inviare il messaggio alla porta corretta. Nel file `clientUDP.c` è necessario modificare il numero di porta di destinazione passato come ultimo argomento alla funzione `UDPSend` (portandolo da 10000 a 100). Nel codice aggiornato, questo si traduce nell'impostare la macro `SERVERPORT` a 100 anche per il client.

- **Per chi usa il proprio PC con Linux... lanciare il server con “`sudo ./serverUDP`” e osservare cosa cambia**

Sui sistemi Linux (e Unix in generale), le porte di rete numerate da 0 a 1023 sono definite "privileged ports" (o well-known ports). Se tenti di eseguire `./serverUDP` in ascolto sulla porta 100 come utente normale, il sistema operativo impedirà l'operazione. La funzione di sistema per riservare la porta (`bind()`) fallirà restituendo un errore di "Permission denied". Di conseguenza, il server non si avvierà.

Utilizzando invece `sudo ./serverUDP`, il programma viene eseguito con i privilegi massimi (utente root), che possiede l'autorizzazione per vincolare i processi alle porte privilegiate (< 1024). In questo caso, il server si avvierà con successo e si metterà normalmente in ascolto sulla porta 100.

4 UDP localhost e hostname

DOMANDA

4) Sostituire “127.0.0.1” prima con “localhost” e poi con “pippo” e osservare cosa succede

RISPOSTA

Ecco cosa succede se nel file `clientUDP.c` modifichiamo l’indirizzo di destinazione (attualmente “127.0.0.1”) all’interno della chiamata a `UDPSend`:

- **Sostituendo con “localhost”:** Non si osserva alcun cambiamento nel comportamento del programma. Il client e il server continueranno a comunicare correttamente. Questo accade perché la funzione di rete utilizzata sotto il cofano dalla libreria (`gethostbyname()`) interroga il sistema di risoluzione dei nomi (DNS locale o file `/etc/hosts`). Il nome “localhost” viene risolto automaticamente nell’indirizzo IP di loopback 127.0.0.1. Di conseguenza, i pacchetti arrivano a destinazione allo stesso identico modo.
- **Sostituendo con “pippo”:** Il client terminerà l’esecuzione con un errore (probabilmente in modo silenzioso o stampando un codice di errore, a seconda di come è gestita la libreria) prima di inviare il messaggio. Questo avviene perché la funzione `gethostbyname("pippo")` tenterà di risolvere l’hostname “pippo” in un indirizzo IP, ma fallirà (poiché “pippo” non è un nome noto nella rete locale, a meno di non averlo configurato manualmente nel file `/etc/hosts`). Andando a ispezionare la libreria `network.c`, si può notare che quando la risoluzione fallisce, la funzione `UDPSend` esegue una `exit(ERRHOST)` terminando brutalmente il processo. Il server, di conseguenza, rimarrà in perenne attesa del messaggio.

5 UDP macchine diverse

DOMANDA

5) [Da fare solo se in Lab Delta] Accordarsi per lavorare su coppie di macchine in modo che server e client siano su macchine diverse. Come bisogna modificare i sorgenti?

RISPOSTA

Per far comunicare server e client posizionati su due macchine fisiche (o virtuali) diverse, l'unica modifica necessaria riguarda il client (`clientUDP.c`). Ecco i dettagli:

- **Modifica al Client (`clientUDP.c`):** Attualmente, la funzione `UDPSend` invia il pacchetto all'indirizzo "127.0.0.1", che rappresenta la macchina locale (loopback interface). Per raggiungere il server su una macchina differente, devi sostituire "127.0.0.1" con il reale indirizzo IP della macchina su cui sta girando il server (ad esempio "192.168.1.50", oppure il corrispondente hostname se la rete lo risolve).

```
// Prima:  
UDPSend(socket, request, strlen(request), "127.0.0.1", 35000);  
  
// Dopo (inserendo l'IP del PC del tuo collega/dell'altra macchina):  
UDPSend(socket, request, strlen(request), "192.168.1.50", 35000);
```

- **Nessuna modifica al Server (`serverUDP.c`):** Il sorgente del server non ha bisogno di essere toccato. Questo perché, se ispezioniamo la libreria `network.c`, la funzione `createUDPInterface` è progettata per configurare il socket (`bind()`) utilizzando la costante `INADDR_ANY`. Questo indica al sistema operativo di mettere il server in ascolto su tutte le interfacce di rete disponibili della macchina (sia quella locale 127.0.0.1 sia l'interfaccia esposta sulla rete LAN associata al suo IP). Pertanto, il server è già pronto a ricevere pacchetti provenienti dall'esterno.

6 UDP riutilizzo porta

DOMANDA

6) Lanciare due volte il server usando due terminali. Cosa si osserva? Funzionano entrambi?

RISPOSTA

Lanciando due volte il server su due terminali diversi per l'esercizio 6, si osserva il seguente comportamento:

- **Si avviano entrambi senza mostrare errori:** Entrambi i processi stamperanno il messaggio "Sono in attesa di richieste..." e non apparirà il tipico errore di porta già occupata (Address already in use). Questo è possibile perché, se andiamo a ispezionare il sorgente della libreria `network.c`, la funzione `createUDPInterface` applica l'opzione `SO_REUSEADDR` al socket prima di chiamare la funzione di sistema `bind()`. Questa opzione comunica al sistema operativo di consentire a più socket di mettersi in ascolto sulla medesima porta.
- **Funzionano entrambi? Solo parzialmente (solo uno riceverà i dati):** Sebbene entrambi siano apparentemente in funzione, quando il client invia un pacchetto UDP a quella porta, il sistema operativo lo consegnerà solo a uno dei due server (generalmente all'ultimo che è stato avviato in ordine temporale, o ripartendo le richieste, a seconda della gestione specifica di rete del kernel Linux). Poiché i messaggi inviati dal client sono indirizzati specificamente alla porta (pacchetti unicast e non multicast o broadcast), il pacchetto non verrà duplicato. L'esito pratico è che un server gestirà la richiesta e l'altro rimarrà in perenne attesa senza ricevere nulla.

7 UDP limite richieste

DOMANDA

- 7) Modificare il server in maniera che soddisfi 5 richieste prima di terminare
- E se volessi che non terminasse mai?

RISPOSTA

Ecco le risposte alle tue domande:

- **Modificare il server per soddisfare 5 richieste prima di terminare:** Nel file `serverUDP.c`, ho sostituito il ciclo di ascolto infinito `while(true)` con un ciclo definito che itera esattamente 5 volte. Il codice del blocco di ascolto è diventato il seguente:

```
// Al posto di while(true)
for(int i = 0; i < 5; i++) {
    printf("[SERVER] Sono in attesa di richieste da qualche client\n");
    UDPReceive(socket, request, MTU, hostAddress, &port);
    // ... stampe ...
    UDPSend(socket, response, strlen(response), hostAddress, port);
}
// Dopo aver servito i 5 client, il programma esce dal loop e il main()
termina.
```

- **E se volessi che non terminasse mai?** Per avere un server che resta in ascolto perenne senza terminare mai (comportamento tipico dei server veri e propri, detti anche daemoni), si utilizza un ciclo infinito. Nel linguaggio C, questo si ottiene utilizzando costrutti condizionali perennemente verificati, come:

- `while(true)` (utilizzato nella versione originale, richiede la libreria `<stdbool.h>`)
- `while(1)` (costrutto tipico in C per indicare che la condizione non sarà mai 0)
- `for(;;)` (ciclo for privo di condizioni di terminazione)

In questo modo il programma attenderà all'infinito un nuovo pacchetto sulla porta senza fermarsi mai, a meno che non venga interrotto forzatamente (es. chiudendo il terminale o premendo `Ctrl+C`).

8 UDP invio numeri

DOMANDA

7) Compilare ed eseguire il secondo esempio

RISPOSTA

Avviando poi in background il server `./serverUDP_inc` e successivamente lanciando il client `./clientUDP_inc`, il comportamento osservato in esecuzione è il seguente:

- Il server si mette in ascolto sulla porta 35000.
- Il client chiede in input un numero intero (es. ho inserito 42).
- Il client invia l'intero al server.
- Il server lo riceve, calcola `request + 1` (ottenendo 43) e lo rispedisce al client.
- **Il client riceve la risposta dal server e stampa a video:** [CLIENT]
Contenuto: 43. Entrambi i programmi, arrivati alla fine, terminano la loro esecuzione.

9 UDP sommatrice

DOMANDA

8) Modificarlo per costruire una semplice sommatrice

- Il client acquisisce ripetutamente da tastiera un numero intero e lo manda al server finché l'utente digita zero
- Il server accumula in una variabile "somma" i valori mandati dal client finché il client manda zero
- Quando il client manda zero il server risponde al client con la somma ottenuta

RISPOSTA

Ho completato la trasformazione del codice in una "sommatrice UDP" operando sui sorgenti presenti nella cartella `es9` e li ho ricompilati.

Ecco i dettagli delle modifiche implementate per soddisfare le specifiche:

- **Modifiche al Client (`clientUDP_inc.c`):** Ho racchiuso l'acquisizione da tastiera e l'invio del pacchetto all'interno di un ciclo `do-while` che ha come condizione di permanenza `request != 0`. In questo modo il client continua a richiedere numeri all'utente e a spedirli al server. Non appena l'utente digita 0, quest'ultimo valore viene inviato, il ciclo si interrompe e il client si mette in attesa (`UDPReceive`) del risultato finale.
- **Modifiche al Server (`serverUDP_inc.c`):** Ho dichiarato una nuova variabile intera `int somma = 0;` per fungere da accumulatore. Anche in questo caso ho introdotto un ciclo `do-while` con la stessa condizione (`request != 0`). Ad ogni iterazione il server riceve un valore, lo stampa e lo somma all'accumulatore (`somma += request;`). Non appena riceve il numero 0 (che non altera la somma), il server esce dal ciclo e sfrutta una `UDPSend` per rispedire il valore finale calcolato (`somma`) al client.

10 UDP sommatrice inaffidabilita

DOMANDA

Sommatrice UDP e perdita di pacchetti

9) Usare la sommatrice su due macchine distinte provando, sulla macchina del client, a staccare il cavo di rete prima di un invio di un dato, ad es.

- Digitare "2345" + INVIO
- Digitare "5187" + INVIO
- Staccare il cavo
- Digitare "2" + INVIO
- Riattaccare il cavo e aspettare 30 sec che il sistema operativo si riassetti
- Digitare "1" + INVIO
- "0"
- Che somma leggo? E' corretta?
- Se il server gira su una macchina Linux (anche una macchina virtuale) e comunica con il client attraverso una rete WiFi oppure in loopback è possibile simulare la perdita di pacchetti bloccando e, dopo poco tempo, sbloccando la ricezione di pacchetti con i seguenti comandi:
 - PER BLOCCARE: `sudo iptables -A INPUT -p udp -dport <porta_del_server> -j DROP`
 - PER SBLOCCARE: `sudo iptables -D INPUT -p udp -dport <porta_del_server> -j DROP`
 - Si consiglia di prepararsi questi comandi in una shell per poterli dare velocemente

RISPOSTA

Questo esercizio dimostra in modo pratico una delle caratteristiche fondamentali del protocollo UDP: la sua inaffidabilità (best-effort delivery) e l'assenza di connessione. Rispondendo alle domande:

- **Che somma leggo?** Leggerai la somma 7533 (derivata da $2345 + 5187 + 1$).
- **È corretta?** Dal punto di vista dell'utente non è corretta, in quanto manca il 2 (la somma attesa sarebbe dovuta essere 7535). Dal punto di vista del protocollo UDP, invece, è un comportamento del tutto normale.

Ecco cosa succede nel dettaglio e il perché di questo risultato:

1. I primi due pacchetti (2345 e 5187) vengono inviati e raggiungono il server, che inizia ad accumulare la somma.
2. Quando scolleghi il cavo (o attivi la regola di iptables per far scartare (DROP) i pacchetti) e digiti 2, la funzione `UDPSend` nel codice del client non restituisce alcun errore. Per il client l'invio ha avuto successo, perché UDP non prevede alcun

meccanismo di conferma di ricezione (ACK). Il pacchetto contenente il "2" viene semplicemente scartato dalla rete e perso per sempre (packet loss).

3. Quando riattacchi il cavo e invii 1 e 0, la rete è di nuovo operativa. Questi pacchetti raggiungono il server che li somma ai precedenti.
4. Quando arriva lo 0, il server risponde inviando il totale accumulato (7533) al client.

Questo esercizio mette in luce che, qualora si utilizzi UDP per applicazioni che necessitano di garanzie di consegna, è necessario implementare i controlli di ricezione, numerazione dei pacchetti e ritrasmissione direttamente a livello applicativo, poiché UDP a livello di trasporto non offre nulla di tutto ciò.

11 UDP sommatrice multicient

DOMANDA

sommatrice UDP e influenze reciproche
10)

- Invocare il server della sommatrice con due client diversi (tutti e tre possono anche essere sulla stessa macchina ovviamente su finestre terminali diverse), ad es.
- Che somma leggo da ciascun client? E' la somma che ciascun client da solo si aspetterebbe?

Client A:

- Digitare "2345" + INVIO
- Digitare "5187" + INVIO
- Digitare "2" + INVIO
- Digitare "1" + INVIO
- "0"

Client B:

- Digitare "2" + INVIO
- Digitare "8" + INVIO
- "0"

RISPOSTA

In questo scenario, il server si comporterà in modo anomalo dal punto di vista applicativo. Ecco le risposte alle due domande:

- **Che somma leggo da ciascun client?** Il client che per primo invia il valore "0" leggerà una somma "mista" composta dai numeri inviati da entrambi i client fino a quel momento. Supponendo che i pacchetti arrivino in modo intervallato, il server sommerà tutti i valori insieme (ad esempio $2345 + 5187 + 2 + 1 + 2 + 8 = 7545$). Il primo client a inviare lo "0" riceverà come risposta 7545 (o un valore intermedio a seconda di quanti pacchetti di B sono arrivati prima del suo "0"). Il secondo client (che manda lo "0" per ultimo) non riceverà nulla se il server termina l'esecuzione, o riceverà una somma azzerata/parziale se il server ha un ciclo esterno che lo fa ricominciare.
- **È la somma che ciascun client da solo si aspetterebbe?** Assolutamente no. Il Client A si aspetterebbe 7535 e il Client B si aspetterebbe 10. Invece, i dati si sono mischiati.

Perché succede questo? A differenza di TCP (che crea un "tubo" dedicato e isolato per ogni connessione/client), UDP non ha il concetto di connessione. Il server ha una sola coda di ricezione per la porta UDP e una singola variabile somma. Quando esegue `UDPReceive`, legge semplicemente il prossimo pacchetto in coda, senza distinguere se proviene dal Client A o dal Client B, sommandolo all'accumulatore globale. Inoltre,

quando il server riceve lo "0" da uno dei due, il ciclo si interrompe e la funzione `UDPSend` risponde all'indirizzo IP e alla porta dell'ultimo pacchetto ricevuto (cioè quello del client che ha mandato lo "0"), inviando il "calderone" di numeri mischiati.

Questo esercizio evidenzia che UDP, non gestendo nativamente le sessioni, richiede allo sviluppatore di identificare esplicitamente da quale client proviene ogni messaggio e di gestire le variabili di stato (come la "somma") separatamente per ogni utente.

12 TCP base

DOMANDA

Esercizi su TCP

11) Lanciare due volte il server usando due terminali. Cosa si osserva? Funzionano entrambi?

RISPOSTA

Essendo ora passati agli esercizi su TCP, il comportamento del programma è differente rispetto al precedente caso UDP.

Lanciando due volte il server TCP (presente nella cartella es12) su due terminali, si osserva quanto segue:

- **Il primo server funziona regolarmente:** si avvia, esegue il `bind()` e si mette in ascolto delle connessioni (`listen()`), stampando il messaggio di attesa a video.
- Il secondo server NON funziona e termina immediatamente.

Perché succede questo? A differenza di quanto visto con UDP, nel protocollo TCP l'opzione del socket `SO_REUSEADDR` ha un significato più stringente: serve esclusivamente a consentire il riutilizzo di una porta che si trova in stato di "TIME_WAIT" (cioè una porta utilizzata da una connessione chiusa di recente dal sistema operativo). Tuttavia, `SO_REUSEADDR` non consente a due programmi di mettersi contemporaneamente in stato di ascolto attivo (`listen`) sulla medesima porta e IP. Di conseguenza, il tentativo di eseguire la `bind()` da parte del secondo server andrà incontro a un errore (del tipo Address already in use) e, come previsto dalla gestione degli errori all'interno della funzione `createTCPServer` nel file `network.c`, causerà la chiusura immediata (terminazione silente tramite `exit()`) del secondo programma.

13 TCP sommatrice

DOMANDA

12) Scrivere la sommatrice usando TCP, compilare ed eseguire

RISPOSTA

Ho completato la scrittura della sommatrice basata su TCP operando sui file nella cartella es13 e li ho ricompilati.

Ecco nel dettaglio cosa ho modificato:

- **Modifiche al Client (`clientTCP.c`):** Subito dopo aver instaurato la connessione logica (con `createTCPConnection`), ho inserito un ciclo `do-while` che continua a chiedere e acquisire un numero intero dall'utente (`scanf`), inviandolo immediatamente al server tramite `TCPSend` sul canale stabilito. Il ciclo termina quando l'utente digita lo 0. Dopodiché, il client si mette in attesa con `TCPreceive` e stampa a video la somma finale prima di chiudere la connessione.
- **Modifiche al Server (`serverTCP.c`):** Subito dopo aver accettato la connessione (`acceptConnection`), ho introdotto la variabile `int somma = 0;`. Da quel momento il server entra in un ciclo `do-while` in cui usa `TCPreceive` per leggere i valori in arrivo dal "tubo" logico del client e li accumula nella variabile `somma`. Non appena arriva il valore 0, il server esce dal ciclo e utilizza la `TCPSend` per rispedire la somma totale al client. Al termine chiude la connessione.

14 TCP sommatrice bloccante

DOMANDA

13) Provare a rifare il caso dell'Esercizio 10 ma con questa nuova versione della sommatrice. Cosa si può osservare? Che soluzione si può trovare? C'è influenza reciproca tra i due client?

RISPOSTA

Rifacendo il caso dell'Esercizio 10 con la nuova versione TCP della sommatrice, ci troveremo di fronte a uno scenario completamente diverso rispetto a UDP.

Ecco cosa succede rispondendo alle tue domande:

- **Cosa si può osservare?** Osserverai che il server gestirà soltanto il Client A, ignorando del tutto il Client B fino alla fine. Quando avvii il server, questo si blocca sulla `acceptConnection()` in attesa del primo client. Avviando il Client A, la connessione viene instaurata e il server entra nel ciclo per calcolare la somma di A. Se nel frattempo avvii il Client B, a livello di rete il sistema operativo del server accetterà "dietro le quinte" la connessione TCP (grazie alla coda di `listen()`), permettendo al Client B di avviarsi e farti inserire i numeri. Tuttavia, il programma del server è bloccato all'interno del ciclo `do-while` a leggere esclusivamente dal canale (socket) del Client A. I numeri inviati dal Client B si accumuleranno semplicemente nel buffer del sistema operativo in attesa di essere letti. Quando il Client A invierà lo "0", il server gli risponderà con la somma corretta (es. 7535), chiuderà la connessione e, per come è scritto attualmente il codice, il programma del server terminerà. Di conseguenza, il Client B non riceverà mai alcuna risposta, la sua connessione verrà interrotta bruscamente dal sistema operativo (generando un errore se prova a inviare o ricevere altro) e i suoi dati andranno persi senza essere stati elaborati.
- **C'è influenza reciproca tra i due client?** Rispetto ai dati, assolutamente no. A differenza di UDP, dove i numeri dei due client finivano mischiati nello stesso "calderone", TCP crea un "tubo" dedicato per ogni connessione. I dati non si sovrappongono in alcun modo e le somme non si inquinano a vicenda. L'unica (pesante) influenza reciproca è di tipo temporale/bloccante: il Client A "monopolizza" il server impedendo che il Client B venga servito (e nel nostro caso, causandone poi l'esclusione definitiva).
- **Che soluzione si può trovare?** Per risolvere il problema ci sono due step:
 - **Per servirli in modo sequenziale (uno dopo l'altro):** È necessario racchiudere la `acceptConnection` e la logica della somma all'interno di un ciclo `while(true)` nel server. In questo modo, dopo aver servito il Client A (alla ricezione dello "0"), il server tornerà in attesa e farà la `accept()` della connessione del Client B che era rimasta "parcheggiata" in coda.
 - **Per servirli in modo concorrente (contemporaneamente):** Serve un'architettura multi-thread (o multi-processo). Il server accetta la connessione nel thread principale e poi passa il socket appena creato a un nuovo thread "figlio" (es. usando la `createConcurrentNetworkFunction` fornita dalla libreria `network.c`). Il thread figlio si occuperà di fare la somma per quel client, mentre il padre tornerà immediatamente in attesa di nuove connessioni (`acceptConnection`) per altri client.

15 TCP sommatrice affidabilità

DOMANDA

14) Usare la sommatrice TCP su due macchine distinte provando, sulla macchina del client, a staccare il cavo di rete prima di un invio di un dato, ad es.

- Digitare "2345" + INVIO
- Digitare "5187" + INVIO
- Staccare il cavo
- Digitare "2" + INVIO
- Riattaccare il cavo e aspettare 30 sec che il sistema operativo si riassetti
- Digitare "1" + INVIO
- "0"

Che somma leggo? E' corretta?

- **NOTA:** Se il server gira su una macchina Linux (anche una macchina virtuale) e comunica con il client attraverso una rete WiFi oppure in loopback è possibile simulare la perdita di pacchetti bloccando e, dopo poco tempo, sbloccando la ricezione di pacchetti con i seguenti comandi:

- PER BLOCCARE: `sudo iptables -A INPUT -p tcp -dport <porta_del_server> -j DROP`
- PER SBLOCCARE: `sudo iptables -D INPUT -p tcp -dport <porta_del_server> -j DROP`
- Si consiglia di prepararsi questi comandi in una shell a parte per poterli dare velocemente

RISPOSTA

A differenza di quanto accaduto nell'analogo esercizio svolto con UDP (Esercizio 9), in questo caso la situazione cambia radicalmente grazie all'affidabilità garantita dal protocollo TCP.

Rispondendo alle tue domande:

- **Che somma leggo?** Leggerai la somma 7535 (che deriva da $2345 + 5187 + 2 + 1$).
- **È corretta?** Sì, è assolutamente corretta. Nessun dato è andato perso.

Cosa è successo dietro le quinte?

1. I primi due invii (2345 e 5187) avvengono normalmente.
2. Quando scolleghi il cavo (o attivi la regola di drop su iptables) e invii il "2", la funzione `TCPSend` accoda il dato nel buffer di trasmissione locale e il sistema operativo prova a spedirlo. Non ricevendo il pacchetto di conferma (ACK) dal server a causa dell'assenza di rete, TCP non scarta il pacchetto, ma lo conserva nel buffer

e avvia un timer. Scaduto il timer, TCP effettuerà automaticamente una serie di ritrasmissioni a intervalli di tempo crescenti.

3. Quando riattacchi il cavo, la connettività si ripristina. Alla successiva ritrasmissione automatica, il pacchetto contenente il "2" raggiungerà finalmente il server (con un leggero ritardo, invisibile all'applicazione se non per l'attesa).
4. Digitando successivamente "1" e "0", anche questi verranno consegnati, sempre garantendo l'ordine corretto di arrivo.
5. Il server riceverà tutti i numeri nella sequenza esatta, senza perdersi alcun pezzo, calcolerà il totale e lo restituirà al client.

A meno che il cavo non rimanga staccato per un tempo lunghissimo (tipicamente svariati minuti, portando la connessione in timeout e causando un errore irreversibile sul socket), TCP è in grado di "riparare" autonomamente questo genere di guasti temporanei di rete senza che lo sviluppatore debba scrivere una singola riga di codice aggiuntiva.

16 TCP trasferimento file

DOMANDA

applicazione di trasferimento file

- Il client chiede al server un file specificandone il nome
- Il server lo trasmette un byte alla volta
- Il client lo salva in locale con lo stesso nome
- Quale protocollo usiamo?
- [Da fare solo con i PC del Lab Delta] Lanciare client e server su due macchine diverse, trasferire un file di grosse dimensioni in modo da avere il tempo di staccare e riattaccare il cavo di rete per un breve tempo durante il trasferimento.

Cosa succede al file trasferito?

RISPOSTA

Applicazione di Trasferimento File TCP (es16)

Ho completato la scrittura e compilazione della versione TCP per il trasferimento file (byte per byte), modificando i sorgenti contenuti nella cartella es16.

Modifiche ai Sorgenti

Client (`clientTCP.c`)

Il client richiede in input all'utente il nome del file da scaricare. Invia il nome al server usando `TCPSend` (inclusendo il carattere terminatore). Apre un file in locale in modalità scrittura binaria ("`wb`"). Entra in un ciclo `while` in cui chiama ripetutamente `TCPReceive(&byte, 1)` per scaricare il file.

NOTE

La ricezione termina automaticamente e correttamente (EOF) nel momento in cui `TCPReceive` restituisce 0, evento che si verifica quando il server chiude la connessione alla fine del trasferimento.

Server (`serverTCP.c`)

Il server accetta la connessione e, per prima cosa, riceve il nome del file (leggendo fino al carattere `\0`). Apre il file richiesto in lettura binaria ("`rb`"). Inizia un ciclo leggendo un byte alla volta dal disco (tramite `fread`) e spedendolo nel tubo TCP usando `TCPSend`. Terminata la lettura dal disco, chiude il file locale e, soprattutto, chiude la connessione (`closeConnection`). In questo modo invia implicitamente al client il segnale di "Fine File".

Tutto il codice è stato compilato con successo nei rispettivi file `clientTCP` e `serverTCP`. Gli eseguibili sono pronti per essere testati per il trasferimento file!

17 UDP calcolo remoto

DOMANDA

Temi d'esame (per ognuno ragionare sulla scelta del protocollo di livello trasporto)

- Implementare un servizio di calcolo remoto: il client spedisce al server in un unico messaggio due numeri interi e il tipo di operatore (somma, sottrazione, moltiplicazione, divisione) e il server, dopo aver svolto il calcolo, restituisce il risultato al client che così termina la sua esecuzione.

RISPOSTA

Piano di Implementazione: Servizio di Calcolo Remoto (es17)

1. Scelta del Protocollo di Trasporto

La traccia chiede di ragionare sulla scelta tra UDP e TCP per un servizio in cui il client invia un unico messaggio con i dati, riceve la risposta e termina.

Scelta ideale: UDP

- **Motivazione:** Il modello di comunicazione richiesto è strettamente di tipo "Richiesta-Risposta" singola. Il payload (due interi e un carattere per l'operatore) è minuscolo e rientra comodamente in un unico datagramma UDP.
- **Efficienza:** Usare TCP richiederebbe un notevole overhead per stabilire la connessione (3-way handshake) e abbatterla, scambiando una decina di pacchetti di controllo solo per inviare due messaggi utili. Con UDP, i pacchetti in gioco sono solo due (uno di andata, uno di ritorno).
- **Risoluzione di inaffidabilità:** Trattandosi di una singola query, se il pacchetto va perso il client potrebbe semplicemente riprovare (con un timeout applicativo), in maniera del tutto analoga a come funziona il protocollo DNS.

Pertanto, l'implementazione proposta utilizzerà UDP.

2. Struttura dei Dati

Definiremo una struct per il messaggio di richiesta:

```
typedef struct {  
    int operando1;  
    int operando2;  
    char operatore; // '+', '-', '*', '/'  
} RichiestaCalcolo;
```

3. Implementazione del Client (clientUDP.c)

- Crea un'interfaccia UDP (es. porta 20000).
- Chiede in input all'utente i due numeri e l'operatore.
- Popola la struct `RichiestaCalcolo`.
- Spedisce la struct al server (`UDPSend`).
- Attende il risultato in ricezione (`UDPReceive`).
- Stampa il risultato e termina.

4. Implementazione del Server (serverUDP.c)

- Crea un'interfaccia `UDP` in ascolto (es. porta 35000).
- Entra in un ciclo infinito di ricezione `UDPReceive` in attesa di richieste.
- Quando riceve una `RichiestaCalcolo`, valuta l'operatore (`switch` statement).
- Calcola il risultato (float o intero, gestiremo le divisioni o daremo un intero).
- Risponde al client inviando il risultato.

18 TCP calcolo remoto

DOMANDA

Temi d'esame (per ognuno ragionare sulla scelta del protocollo di livello trasporto)

- Implementare un servizio di calcolo remoto: il client spedisce al server in 3 distinti messaggi due numeri interi e il tipo di operatore (somma, sottrazione, moltiplicazione, divisione) e il server, dopo aver svolto il calcolo, restituisce il risultato al client che così termina la sua esecuzione.

RISPOSTA

Piano di Implementazione: Servizio di Calcolo Remoto (es18)

1. Scelta del Protocollo di Trasporto

La traccia chiede di ragionare sulla scelta tra UDP e TCP per un servizio in cui il client invia i dati del calcolo suddivisi in 3 messaggi distinti (operando 1, operatore, operando 2).

Scelta obbligata (o fortemente raccomandata): TCP

- **Motivazione (Ordine di consegna):** Poiché i dati viaggiano su 3 messaggi separati, l'ordine in cui arrivano al server è fondamentale. UDP non garantisce la consegna in ordine: il server potrebbe ricevere prima l'operatore e poi il primo numero, confondendo la logica di ricezione. TCP, essendo orientato allo stream, garantisce la consegna ordinata.
- **Motivazione (Affidabilità e Sessione):** Se un pacchetto UDP andasse perso, il server rimarrebbe bloccato in attesa di un "pezzo" del calcolo mancante (o peggio, leggerebbe il pezzo della richiesta di un altro client). TCP garantisce che, finché la connessione è aperta, i 3 messaggi arriveranno a destinazione completi. Inoltre, la connessione TCP fornisce un "tubo dedicato" (sessione) per il client: il server non rischia di mischiare il primo operando del Client A con il secondo operando del Client B.

Pertanto, l'implementazione proposta utilizzerà TCP.

2. Implementazione del Client (`clientTCP.c`)

- Instaura una connessione TCP con il server (porta 35000).
- Chiede in input all'utente il primo operando, l'operatore e il secondo operando.
- Effettua tre distinte chiamate `TCPSend`:
 - Invia operando1 (int)
 - Invia operatore (char)
 - Invia operando2 (int)
- Effettua una `TCPReceive` per attendere il risultato (float).
- Stampa il risultato e chiude la connessione.

3. Implementazione del Server (`serverTCP.c`)

- Crea un server TCP in ascolto sulla porta 35000.

- In un ciclo infinito `while(1)`, accetta una nuova connessione (`acceptConnection`).
- Effettua tre distinte chiamate `TCPReceive` sequenziali dal socket del client:
 - Riceve operando1 (int)
 - Riceve operatore (char)
 - Riceve operando2 (int)
- Calcola il risultato con uno `switch` (gestendo la divisione per zero).
- Usa `TCPSend` per inviare il risultato al client.
- Chiude il socket del client (`closeConnection`) e torna in ascolto per il prossimo client.

19 TCP servizio telepass

DOMANDA

Temi d'esame (per ognuno ragionare sulla scelta del protocollo di livello trasporto)

- Implementare un servizio di telepass: quando l'auto si avvicina al casello il terminale in esso presente spedisce al server il numero di targa; il server tiene traccia in una struttura dati del numero di passaggi per ogni targa e restituisce al client tale numero.

RISPOSTA

Piano di Implementazione: Servizio Telepass (es19)

1. Scelta del Protocollo di Trasporto

La traccia richiede di inviare un numero di targa (stringa) al server, il quale deve incrementare un contatore per quella specifica targa e restituire il numero aggiornato di passaggi.

Scelta raccomandata: TCP

- **Motivazione (Affidabilità e Modifica di Stato):** Il problema principale di questo scenario è che il server modifica uno stato interno (incrementa il numero di passaggi, che ha rilevanza per la fatturazione/tracciamento). Se usassimo UDP e il pacchetto di risposta (dal server al client) andasse perso, il client andrebbe in timeout e ritrasmetterebbe la richiesta. Il server, ricevendo nuovamente il datagramma UDP, incrementerebbe il contatore per la seconda volta per lo stesso passaggio fisico dell'auto, creando un'incongruenza grave (doppio addebito).
- **Vantaggio di TCP:** Usando TCP, la libreria di rete gestisce internamente le ritrasmissioni e l'eliminazione dei pacchetti duplicati. Il server è garantito di ricevere la richiesta una e una sola volta per ogni connessione stabilita dal casello per un dato veicolo. Se c'è un problema di rete, la connessione cade, ma non si creano addebiti "fantasma" dovuti a ritrasmissioni applicative non gestite.
- **Nota:** Si potrebbe usare UDP solo a patto di implementare a livello applicativo un identificatore univoco di transazione (ID Passaggio) per riconoscere i duplicati, ma in un contesto base TCP offre questa garanzia "gratis".

Pertanto, l'implementazione proposta utilizzerà TCP.

2. Implementazione del Client (`clientTCP.c`)

- Il client (Terminale Casello) instaura una connessione TCP con il server.
- Legge o simula l'acquisizione della targa dell'auto (es. `char targa[10]`).
- Invia la stringa della targa al server tramite `TCPSend`.
- Effettua una `TCPReceive` per attendere in risposta l'intero relativo al conteggio dei passaggi.
- Stampa il risultato (es. "L'auto AB123CD è passata 4 volte") e chiude la connessione.

3. Implementazione del Server (`serverTCP.c`)

- Crea un server TCP in ascolto.

- Gestisce una semplice struttura dati (es. un array di **struct** contenenti targa e conteggio, per semplicità limitato a un numero massimo di targhe).
- In un ciclo infinito, accetta una connessione da un casello (**acceptConnection**).
- Riceve la targa (**TCPReceive**).
- Cerca la targa nella struttura dati:
 - Se esiste, incrementa il conteggio.
 - Se non esiste, la aggiunge con conteggio = 1.
- Risponde al client inviando il conteggio aggiornato (**TCPSend**).
- Chiude il socket del client e torna in attesa del prossimo passaggio.

20 TCP terminale remoto

DOMANDA

Temi d'esame (per ognuno ragionare sulla scelta del protocollo di livello trasporto)

- Implementare una semplice versione di terminale remoto. Il client si connette ad un server di cui si conosce IP e porta e chiede all'utente di scrivere un comando come in un normale terminale a caratteri (detto anche shell o console) di Linux. Il comando viene eseguito dal server e il relativo output viene stampato a video dal client.

RISPOSTA

Piano di Implementazione: Terminale Remoto (es20)

1. Scelta del Protocollo di Trasporto

La traccia richiede che il client si connetta a un server, invii un comando (es. comando di shell) e riceva l'output risultante dall'esecuzione di quel comando dal server.

Scelta raccomandata: TCP

Motivazione:

In questo scenario è fondamentale che i dati non vadano persi né vengano ricevuti in ordine errato. Un comando potrebbe risultare corrotto (ad es. un "ls -l" potrebbe arrivare incompleto), ma l'aspetto critico è la ricezione dell'output. L'output di un comando di shell può essere di dimensioni arbitrarie e superare ampiamente il limite di singolo pacchetto (MTU) supportato tipicamente da UDP. Il protocollo TCP garantisce una trasmissione orientata al flusso (stream) e gestisce nativamente la segmentazione e la ritrasmissione dei pacchetti qualora essi si perdano nella rete.

Il server leggerà l'output tramite una pipe (ad es. usando `popen` in C) e lo invierà sul socket TCP in vari blocchi. Il client continuerà semplicemente a leggere dal socket finché non riceverà un segnale di termine (la chiusura della connessione o fine dei dati). Con UDP, al contrario, sarebbe necessario implementare a livello applicativo la frammentazione dell'output e gestire esplicitamente ACK/NACK per garantire che il client riceva tutto l'output ordinato correttamente.

2. Implementazione

- Il client TCP legge il comando inserito dall'utente (es. tramite `fgets`), lo invia al server tramite `TCPSend`, e poi entra in un ciclo in cui legge (`TCPReceive`) l'output inviato dal server e lo stampa a video, fermandosi quando la trasmissione è completata (chiusura del socket o invio terminato).
- Il server TCP accetta connessioni. Quando riceve un comando, usa la chiamata di sistema "`popen(comando, "r")`" per eseguirlo come comando di shell e aprire una pipe per leggerne l'output. Legge l'output a blocchi usando `fread` e li invia man mano al client tramite `TCPSend`. Finito l'output, chiude il file associato a `popen` (`pclose`) e chiude il socket, il che indicherà al client che il risultato del comando è stato interamente trasmesso.

21 Web polling

DOMANDA

Esercizio

- Scrivere e provare una pagina HTML che ricarica periodicamente il sito dell'ANSA

RISPOSTA

Soluzione Esercizio 21: Ricaricamento periodico sito ANSA (Polling)

Per ricaricare periodicamente il sito, si è utilizzata la funzione JavaScript `setInterval()` nel file `javascript-load.html`. Questo script aggiorna l'attributo `src` dell'iframe a intervalli regolari (es. ogni 10 secondi). Inoltre, aggiungendo un parametro "timestamp" all'URL, si evita che il browser mostri la pagina dalla cache locale, forzando un vero reload dal server dell'ANSA.

Approccio sincrono al refresh (Polling)

Questo metodo prende il nome di "polling". Il client effettua richieste periodiche al server in base a un timer fisso, comportandosi in modo "sincrono" rispetto alla scadenza di questo timer.

Limiti e problematiche del Polling:

1. **Spreco di risorse (Refresh inutili):** Se il server dell'ANSA non ha pubblicato nuove notizie negli ultimi 10 secondi, la richiesta è inutile. Si spreca larghezza di banda sul lato client e capacità di elaborazione/rete sul lato server, che invierà dati identici ai precedenti.
2. **Ritardo negli aggiornamenti (Mancato refresh temporaneo):** Se una notizia urgente (breaking news) viene pubblicata subito dopo che l'iframe è stato ricaricato, l'utente dovrà attendere l'intero ciclo del timer (altri 10 secondi) prima di poterla leggere.

L'Alternativa: Approccio Asincrono (Event-Driven)

L'alternativa è la programmazione orientata agli eventi. Utilizzando tecnologie come le Web Socket (o i Server-Sent Events, SSE), il client stabilisce una connessione persistente e bidirezionale col server. In questo modo, il client non ha bisogno di chiedere continuamente "Ci sono novità?". Sarà invece il server, nel momento esatto in cui viene pubblicata una nuova notizia, a "spingere" (push) in modo asincrono l'aggiornamento verso il client connesso.

Questo risolve entrambi i problemi: azzera il consumo di banda per richieste a vuoto e permette un aggiornamento dei contenuti rigorosamente in tempo reale.

22 Web server

DOMANDA

Un semplice web server

- Aprire il file `serverHTTP.c` in `Esempi-web/` e analizzarne il contenuto
- Compilarlo come nell'esercitazione sull'interfaccia socket ed eseguirlo
- Provare ad aprire un secondo terminale nella stessa cartella e a rilanciare lo stesso server. Funziona? Perché?
- Aprire il browser preferito e impostare la URL `"http://127.0.0.1:8000/"` Cosa si vede sul browser e sul terminale?
- Aprire il browser preferito e impostare la URL `"http://localhost:8000/"` Cosa cambia?

RISPOSTA

Soluzione Esercizio 22: Un semplice web server

Operazioni Preliminari e Compilazione

Inizialmente sono stati copiati i file necessari (`serverHTTP.c`, `network.c`, `network.h`) all'interno della cartella dell'esercizio. Analizzando il sorgente di `"serverHTTP.c"`, sono stati corretti alcuni commenti non validi scritti in stile Python (carattere `"#"`) sostituendoli con quelli corretti in linguaggio C (`"//"`). Infine, il codice è stato compilato tramite il comando `gcc`.

Il file è un banale server TCP che resta in ascolto sulla porta 8000, accetta la connessione di un client, processa testualmente la richiesta leggendo riga per riga l'header HTTP, invia in risposta una pagina HTML di base e infine chiude la socket.

Rilancio del Server su un secondo terminale

Se si prova a lanciare una seconda istanza del server in un altro terminale, mentre il primo è ancora in esecuzione, il programma terminerà subito per un errore. Questo avviene a causa del fallimento della syscall `"bind"`: non è possibile associare la medesima porta e lo stesso indirizzo a due processi in ascolto nello stesso istante.

Navigazione tramite l'indirizzo IP (127.0.0.1)

Richiedendo con il browser la URL `"http://127.0.0.1:8000/"`:

- **Nel browser:** Si visualizza il contenuto statico inserito nel server (la scritta `"Hello World, this is a very simple HTML document."`).
- **Nel terminale:** Il programma stampa la richiesta HTTP pura inviata dal browser. Tra le varie intestazioni è presente la prima riga `"GET / HTTP/1.1"` e in particolare l'intestazione `"Host: 127.0.0.1:8000"`.

Navigazione tramite Hostname (localhost)

Richiedendo con il browser la URL `"http://localhost:8000/"`:

- **Nel browser:** Non cambia assolutamente nulla, in quanto l'hostname `"localhost"` viene risolto localmente nell'IP di loopback `127.0.0.1`. Il server riceverà la connessione e invierà in risposta la stessa identica pagina HTML.
- **Nel terminale:** La richiesta catturata cambierà l'intestazione `Host`, che rifletterà la stringa usata dal client, diventando `"Host: localhost:8000"`. Questo dimostra

come l'hostname testuale digitato dall'utente venga inserito dal browser e trasmesso all'interno delle intestazioni HTTP.

Comandi Utili per il Test e l'Analisi

```
# 1. Compilazione
gcc serverHTTP.c network.c -o serverHTTP

# 2. Esecuzione del Web Server
./serverHTTP

# 3. Test tramite CLI (utilizzando curl per visualizzare gli header scambiati)
curl -v http://127.0.0.1:8000/
curl -v http://localhost:8000/

# 4. Registrazione del Traffico (cattura su interfaccia locale 'lo' sulla porta 8000)
# Tramite riga di comando (salvataggio su file .pcap):
sudo tshark -i lo -f "tcp port 8000" -w web_traffic.pcap

# 5. Analisi del Traffico Registrato
# --- Tramite Riga di Comando (CLI) ---
# 1. Leggere tutti i pacchetti registrati:
tshark -r web_traffic.pcap

# 2. Filtrare per mostrare solo i messaggi a livello applicativo HTTP:
tshark -r web_traffic.pcap -Y "http"

# 3. Leggere il contenuto completo delle intestazioni (Dettagli pacchetto estesi):
tshark -r web_traffic.pcap -Y "http" -V

# 4. Seguire l'intera conversazione scambiata:
tshark -r web_traffic.pcap -z follow,tcp,ascii,0

# --- Tramite Interfaccia Grafica (GUI) ---
# Lanciare Wireshark normalmente:
sudo wireshark
# Oppure per aprire direttamente il file registrato per l'analisi visiva:
wireshark web_traffic.pcap
```

23 Web form GET

DOMANDA

form-get.html

```
<!DOCTYPE html><html>
<body>
<h2>The method Attribute</h2>
<p>This form will be submitted using the GET method:</p>
<form action="/action" target="_blank" method="get">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
</form>
<p>After you submit, notice that the form values is visible in the address bar
of the new browser tab.</p>
</body>
</html>
```

Esercizio

- Eseguire il server web `serverHTTP.c` e con il browser preferito aprire il file `form-get.html`
- Cosa si vede?
- Provare ad analizzare il contenuto della connessione TCP con Wireshark.
- Cosa si vede?

RISPOSTA

Soluzione Esercizio 23: Web form GET

Operazioni Preliminari

1. È stato copiato il server in linguaggio C e il codice HTML per il modulo web forniti dal professore.
2. Affinché l'invio del modulo comunichi effettivamente con il server in ascolto in locale, è necessario che il file HTML "`form-get.html`" punti al server giusto. Se il file viene aperto dal browser come file locale, il campo `action="/action"` andrebbe modificato in `action="http://127.0.0.1:8000/action"` (oppure `http://localhost:8000/action`).

Risposta alla Domanda 1: "Cosa si vede (sul browser)?"

Cliccando sul pulsante "Submit", il browser raccoglie i dati inseriti nei campi di input (`fname` e `lname`) e li appende direttamente all'URL di destinazione indicato nell'attributo "action", separandoli con il carattere "?" e unendo le varie coppie chiave-valore con la "&". Nella barra degli indirizzi della nuova scheda che si apre vedremo quindi una stringa simile a questa:

`http://127.0.0.1:8000/action?fname=John&lname=Doe`

A schermo si vedrà la pagina statica HTML di cortesia ("Hello World...") restituita dal nostro `serverHTTP`.

Risposta alla Domanda 2: "Cosa si vede (analizzando il TCP con Wire-

shark)?"

Catturando il traffico sull'interfaccia di loopback, ad esempio tramite il comando:
`sudo tshark -i lo -f "tcp port 8000" -Y "http" -V`
oppure seguendo lo stream TCP nella GUI di Wireshark, si osserverà la richiesta HTTP in chiaro, che mostrerà quanto segue:

```
GET /action?fname=John&lname=Doe HTTP/1.1
Host: 127.0.0.1:8000
User-Agent: ...
```

- Si nota chiaramente che la particolarità del metodo GET per l'invio dei form consiste nel passare i parametri direttamente nell'URL (Request URI).
- Di conseguenza, NON è presente un corpo del messaggio (Body o payload aggiuntivo nella richiesta HTTP).
- Questo metodo espone i dati inseriti direttamente nel log delle richieste del server e nei log dei sistemi intermedi o cronologie del browser, rendendolo completamente inadatto al trasferimento di dati sensibili (come le password).

Comandi Utili per Riprodurre l'Esercizio:

```
# Eseguire il server
./serverHTTP

# Per testare l'invio via terminale e simulare il submit del form:
curl -v "http://127.0.0.1:8000/action?fname=John&lname=Doe"

# Analizzare l'esatta richiesta arrivata usando TShark:
sudo tshark -i lo -f "tcp port 8000" -z follow,tcp,ascii,0
```

24 Web form POST

DOMANDA

form-post.html

```
<!DOCTYPE html><html>
<body>
<h2>The method Attribute</h2>
<p>This form will be submitted using the GET method:</p>
<form action="/action" target="_blank" method="post">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
</form>
<p>After you submit, notice that the form values is visible in the address bar
of the new browser tab.</p>
</body>
</html>
```

Esercizio

- Eseguire il server web `serverHTTP.c` e con il browser preferito aprire il file `form-post.html`
- Cosa si vede?
- Provare ad analizzare il contenuto della connessione TCP con Wireshark.
- Cosa si vede?

RISPOSTA

Soluzione Esercizio 24: Web form POST

Operazioni Preliminari

1. Sono stati nuovamente copiati i file base del server (`serverHTTP.c`) e creata la cartella per l'esercizio.
2. È stato creato il file `"form-post.html"` modificando l'URL di destinazione all'interno dell'attributo `action`, impostandolo a `"http://127.0.0.1:8000/action"` affinché punti al web server locale da noi lanciato. (Nota: Il testo pre-impostato all'interno del file HTML del professore conteneva un refuso, dicendo erroneamente che i valori sarebbero stati visibili nella barra degli indirizzi, ma essendo un POST questo non è vero!).

Risposta alla Domanda 1: "Cosa si vede (sul browser)?"

Dopo aver cliccato il pulsante "Submit", il browser naviga verso la pagina di destinazione (restituendo il consueto messaggio "Hello World").

La differenza fondamentale rispetto all'esercizio precedente è nella BARRA DEGLI INDIRIZZI: questa volta l'URL si presenterà "pulito", mostrando esclusivamente `http://127.0.0.1:8000/action`.

Nessun dato (nome e cognome) viene appeso all'URL, i parametri sono completamente invisibili.

Risposta alla Domanda 2: "Cosa si vede (analizzando il TCP con Wire-

shark)?"

Catturando la connessione su Wireshark (o analizzando l'output che viene stampato sul terminale dal nostro server C), notiamo la struttura della richiesta HTTP POST:

```
POST /action HTTP/1.1
Host: localhost:8000
User-Agent: ...
Content-Length: 20
Content-Type: application/x-www-form-urlencoded

fname=John&lname=Doe
```

A differenza del GET, qui possiamo vedere:

1. Il metodo HTTP è cambiato in POST.
2. I parametri (`fname=John&lname=Doe`) non sono più nella prima riga dell'URI, ma si trovano isolati all'interno del "Corpo" (Body/Payload) della richiesta, separati dalle intestazioni tramite una riga vuota.
3. Proprio a causa della presenza del Body, il client (il browser) ha dovuto specificare al server la dimensione in byte di quest'ultimo usando l'intestazione `Content-Length: 20` (che è il numero esatto di caratteri di `"fname=John&lname=Doe"`) e il tipo di codifica `Content-Type: application/x-www-form-urlencoded`.

Guardando il codice `serverHTTP.c`, si nota proprio la presenza di una logica `"if(strcmp(method, "POST")==0)"` che intercetta i POST e utilizza il valore catturato da `"Content-Length"` per capire quanti caratteri ("byte") leggere per poter estrapolare dal corpo della richiesta l'intero modulo compilato dall'utente.

Comandi Utili per Riprodurre l'Esercizio:

```
# Compilare ed eseguire il server
gcc serverHTTP.c network.c -o serverHTTP
./serverHTTP

# Per testare l'invio via terminale e simulare l'HTML form con metodo POST:
curl -v -d "fname=John&lname=Doe" "http://localhost:8000/action"

# Wireshark (Filtrare per traffico HTTP sulla porta 8000 e guardare la stringa
  TCP):
sudo tshark -i lo -f "tcp_port_8000" -z follow,tcp,ascii,0
```

25 Web server URL

DOMANDA

Esercizio

- Modificare il file `serverHTTP.c` in modo che, invece di restituire sempre la solita pagina web di prova, restituisca una delle pagine html usate dal lucido 9 in poi scelta attraverso l'uso del browser.
- Suggerimenti:
 - Provare a fare la nuova richiesta col browser usando il `serverHTTP.c` in modo da vedere la richiesta HTTP per capire dove si trova la stringa con il nome del file nella richiesta che fa il browser (aiutarsi anche con Wireshark)
- Cosa devo scrivere nella barra del browser?
 - Capire come recuperare la stringa con il nome del file dalla richiesta HTTP (si veda `esempio-parser.c`)
 - Per costruire la risposta riciclare parte del codice usato nell'esercizio del trasferimento di file
- Prova finale: cosa succede se chiedo al server di restituire i file `form-get.html` e `form-post.html`?

RISPOSTA

Soluzione Esercizio 25: Web server e file dinamici (URL)

Operazioni Preliminari e Modifica del Codice

Per questo esercizio ho creato la cartella e ho modificato in modo sostanziale il file `"serverHTTP.c"`:

1. Utilizzando la funzione `strtok` (come suggerito nel file `esempio-parser.c`), ho estratto la seconda parola della prima riga della richiesta HTTP, ovvero l'URI (es: `"/prova.html"`).
2. Ho manipolato questa stringa per rimuovere lo slash `"/"` iniziale e ho inserito una logica che ripulisce la stringa da eventuali "Query String" (es. rimuovendo tutto ciò che c'è da un eventuale punto interrogativo `"?"` in poi).
3. Utilizzando la funzione `fopen`, il server tenta di aprire il file. Se il file esiste, invia prima di tutto gli Header di successo (HTTP/1.1 200 OK) e poi, tramite un ciclo `fgetc`, legge e trasmette al client l'intero contenuto del file. Se il file non esiste, il server restituisce una pagina d'errore dinamica (HTTP/1.1 404 Not Found).

Risposta alla Domanda 1: "Cosa devo scrivere nella barra del browser?"

Per poter richiedere una pagina specifica presente nella cartella Esempi-web (che abbiamo copiato all'interno dell'esercizio), è necessario digitare l'indirizzo base del server aggiungendo lo slash e il nome esatto del file.

Ad esempio: `http://localhost:8000/prova.html` oppure `http://localhost:8000/javascript.html`.

Risposta alla Domanda 2: "Cosa succede se chiedo al server di restituire i file `form-get.html` e `form-post.html`?"

Scrivendo l'indirizzo `http://localhost:8000/form-get.html`:

1. Il browser effettua una richiesta HTTP GET al nostro server C.
2. Il server C individua il file sul disco, ne legge il contenuto testuale (che è formato da tag <form>, <input>, ecc.) e lo rispedisce intatto al browser.
3. Il browser, ricevendo il file, lo **renderizza graficamente**, mostrando all'utente la classica maschera per inserire nome e cognome.
4. Se l'utente compila la form e preme il pulsante "Submit" (e l'azione della form è `action="/action"` come in originale), il browser avvierà autonomamente una **seconda** richiesta HTTP (GET `/action?fname=...`) verso il nostro server C. Poiché il file "action" non esiste, in questo caso il nostro server risponderà con un Errore 404 Not Found!

Questo ci permette di capire a fondo come comunicano Front-end e Back-end: il server prima serve l'interfaccia utente (HTML) su richiesta del client, e successivamente il client usa quell'interfaccia per inviare comandi (dati e parametri) allo stesso server.

Comandi Utili per testare l'Esercizio:

```
# Compila ed esegui il server
gcc serverHTTP.c network.c -o serverHTTP
./serverHTTP

# Richiedi la pagina prova.html da riga di comando per testarlo (oppure apri l'
  URL sul browser):
curl -v http://localhost:8000/prova.html
curl -v http://localhost:8000/file_inesistente.txt
```

26 Upload file server web

DOMANDA

Esercizio per casa

- Modificare il server web `serverHTTP.c` in modo che accetti con il metodo `POST` un intero file da salvare nella cartella del server (il cosiddetto “upload sul server”).
- Suggerimenti:
 - utilizzare il file `form-file.html` con `serverHTTP.c` non modificato ed analizzare il contenuto della connessione `TCP` con `Wireshark` in modo da capire nella richiesta `HTTP`:
 - * Dove si trova il nome del file
 - * Dove si trova il contenuto del file
 - Nella lettura della richiesta `HTTP` sul server aggiungere il codice che salva il file prendendo spunto dal codice usato nell’esercizio del trasferimento di file.
 - Invece la risposta `HTTP` può essere molto statica come nella versione originale di `serverHTTP.c`.

RISPOSTA

Soluzione Esercizio 26: Upload file server web

Operazioni Preliminari

Ho copiato la versione "base" (dell’esercizio 22) di `serverHTTP.c`, insieme ai moduli di rete, nella cartella `es26 - Upload file server web`.

Successivamente ho copiato `form-file.html`, il cui tag `form` utilizzava già il metodo "post" e, soprattutto, l’attributo `enctype="multipart/form-data"` (fondamentale per permettere al browser di trasmettere file anziché semplice testo url-encoded). L’attributo `action` puntava già correttamente a `http://localhost:8000/action`.

Risposta alle domande sull’analisi con `Wireshark`:

Analizzando la richiesta `HTTP` generata dall’inizio del file (tramite `TShark` o l’interfaccia grafica), si nota che l’architettura della richiesta è molto diversa da un normale `POST` testuale. Il "Body" (Corpo) della richiesta è diviso in "parti", separate da una stringa casuale chiamata "boundary" (es. -----SgojsDX1...).

1. Dove si trova il nome del file?

Il nome originario del file caricato dall’utente si trova all’interno del Payload (nel Body della richiesta `HTTP`), annidato in una sotto-intestazione specifica chiamata `Content-Disposition`.

Si presenterà come: `Content-Disposition: form-data; name="myfile"; filename="nome_del_mio_file.txt"`

2. Dove si trova il contenuto del file?

Il contenuto effettivo (i byte del file) si trova subito sotto questa intestazione interna, separato da un’ulteriore riga vuota. Il contenuto termina non appena si incontra la stringa di chiusura del boundary.

Modifiche al server in C

Per gestire l’upload "spartano", ho modificato `serverHTTP.c` prendendo spunto dall’esercizio del trasferimento file.

Nella porzione in cui viene intercettato il metodo POST (`if(strcmp(method, "POST")==0)`), ho aggiunto:

1. L'apertura in scrittura (modalità "wb" - **w**rite **b**inary) di un file temporaneo `file_caricato_dal_client.bin`.
2. Un ciclo `for` che usa `fputc` per travasare tutto ciò che arriva dalla socket (`fgetc(connfd)`) direttamente dentro al file, per un numero di cicli pari all'intestazione HTTP `Content-Length`.
3. Al termine della lettura, il file viene chiuso e salvato sul disco.

N.B: Poiché stiamo scaricando l'intero Payload grezzo direttamente nel file, quest'ultimo conterrà non solo i dati del file trasmesso, ma anche i boundary testuali e le intestazioni interne aggiunte dal browser. Implementare un parser "multipart" corretto in linguaggio C (che rimuova il boundary e i meta-dati per salvare solo il file puro) richiede decine di righe di logica complessa che esulano dagli scopi introduttivi di questa esercitazione. Il meccanismo implementato mostra in modo evidente l'avvenuto "travaso" dei dati dal client al filesystem del server!

Comandi utili:

```
# Compila e avvia il server
gcc serverHTTP.c network.c -o serverHTTP
./serverHTTP

# Esegui curl per caricare (upload) un file locale ("testfile.txt")
# Il parametro -F innesca l'upload form-data
curl -v -F "myfile=@testfile.txt" http://localhost:8000/action
```

27 Web server CGI

DOMANDA

Esempio di web server esteso con gestione CGI

- Aprire il file `serverHTTP-CGI.c` in Esempi-web/ e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
- Aprire il browser preferito e il file `sommatrice-web.html`
 - Cosa si vede sul browser?
 - NOTA: Provare con numeri positivi, negativi, con parte decimale...
- A cosa corrisponde il secondo parametro della funzione `sommatrice()` ?

RISPOSTA

Soluzione Esercizio 27: Web server con gestione CGI

Operazioni Preliminari

Ho copiato i file `serverHTTP-CGI.c` e `sommatrice-web.html` nella nuova cartella dell'esercizio e ho compilato l'eseguibile del server C, per poi avviarlo.

Risposte alle domande:

1. Cosa si vede sul browser aprendo `sommatrice-web.html`?

Si presenta una semplice interfaccia (creata tramite Web Form) che richiede l'inserimento di "Operando 1" e "Operando 2", accompagnata dal pulsante "Somma". Inviando il modulo, il browser invia i dati inseriti appendendoli all'URL (es: `/sommatrice?op1=5.5&op2=-2.2`).

Provando a inviare numeri positivi, negativi o con la virgola (utilizzando il punto come separatore decimale), il server restituisce correttamente il risultato matematico elaborato (es. Risultato=3.300000).

Questo avviene perché il backend scritto in C utilizza la funzione `atof()`, la quale supporta nativamente il parsing delle stringhe contenenti segni negativi e punti decimali.

2. A cosa corrisponde il secondo parametro della funzione `sommatrice()`?

Analizzando la firma della funzione: `void sommatrice(char *url, FILE *out)`

Il parametro `FILE *out` corrisponde allo stream di output del socket TCP, ovvero la connessione aperta con il client (il browser).

Se guardiamo il `main()`, notiamo che la funzione viene richiamata passando `connfd` (il file descriptor della connessione): `sommatrice(url, connfd);`.

Questo meccanismo è l'essenza dell'interfaccia CGI (Common Gateway Interface): il server "delega" l'elaborazione a un programma o funzione esterna, passandole i parametri e fornendole l'accesso diretto allo standard output della rete. Qualsiasi cosa la funzione `sommatrice` stampi tramite `fprintf(out, ...)`, andrà a finire dritta al browser (ecco perché la funzione si preoccupa di stampare lei stessa l'intestazione HTTP 200 OK e i tag HTML).

Comandi utili:

```
# Compila ed esegui
```



```
gcc serverHTTP-CGI.c network.c -o serverHTTP-CGI
./serverHTTP-CGI

# Simula l'inserimento di numeri decimali e negativi (test)
curl -v "http://localhost:8000/sommatrice?op1=5.5&op2=-2.2"
```

28 Web Socket Chat

DOMANDA

Esercizio su Web Socket

- Realizzazione di una chat eseguita dentro un browser web
 - Fare riferimento al materiale apposito condiviso con gli studenti

RISPOSTA

Soluzione Esercizio 28: Web Socket Chat

Operazioni Preliminari

Ho copiato l'intera cartella `WebSocket_Chat` fornita nel materiale didattico all'interno della directory `es28 - Web Socket Chat`. Questo progetto non è scritto in C, bensì sfrutta Node.js e la libreria `Socket.IO` per fornire funzionalità Real-Time.

Analisi dell'Architettura

L'applicazione è composta da due parti fondamentali che comunicano tra loro:

1. Backend (Server in Node.js):

- Il file `server.js` utilizza il framework `express` per avviare un server in ascolto sulla porta 4000.
- La prima cosa che fa il server è servire i file statici (l'interfaccia utente) presenti nella cartella `public`.
- Successivamente, aggancia a questo server il modulo `socket.io`. Da questo momento il server è in grado di accettare richieste di "Protocol Upgrade": quando un client lo richiede, la connessione HTTP standard viene promossa a connessione WebSocket (un tunnel TCP persistente e bidirezionale).
- Quando il server riceve un evento chiamato `message` da un qualsiasi client, usa l'istruzione `io.sockets.emit('UploadChat', data)` per prendere quel messaggio e "spargere la voce" (Broadcast) ritrasmettendolo istantaneamente a TUTTI i client collegati.

2. Frontend (Client in Javascript):

- All'interno di `public/chat.js`, il client apre una connessione WebSocket verso il server: `io.connect('http://localhost:4000')`.
- Quando l'utente preme il tasto per inviare, il browser "emette" l'evento `message` verso il server.
- Contestualmente, il browser si mette "in ascolto" di eventuali eventi `UploadChat` provenienti dal server. Appena ne riceve uno, il javascript manipola il DOM (aggiungendo codice HTML) per mostrare a schermo il nuovo messaggio ricevuto, aggiornando la chat in tempo reale senza dover ricaricare la pagina!

Comandi per testare la Chat:

```
# Posizionati nella cartella
cd "es28-Web_Socket_Chat"

# Installa le dipendenze (se non sono già presenti nella cartella node_modules)
```

```
npm install
```

```
# Avvia il server  
node server.js
```

```
# Ora apri il tuo browser preferito e apri DUE schede diverse puntandole a:  
http://localhost:4000
```

```
# Prova a scrivere un messaggio in una scheda e vedrai che comparirà  
magicamente in tempo reale anche sull'altra!
```

29 Webservice

DOMANDA

Esercizio

- Considerare la cartella Webservice/
- Aprire il file `serverHTTP-REST.c` e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
- Aprire il file `clientREST-GET.c` e analizzarne il contenuto
- Compilarlo come al solito ed eseguirlo
 - Che parametri devo passare in linea di comando?
 - Cosa si può vedere analizzando lo scambio di dati tramite Wireshark?
 - Qual è la signature della funzione `calcolaSomma()` sul server e sul client? Perché ha senso che siano uguali?

RISPOSTA

Soluzione Esercizio 29: Webservice REST (RPC via HTTP)

Operazioni Preliminari

Ho copiato i file `serverHTTP-REST.c`, `clientREST-GET.c` e la libreria di rete all'interno della cartella dell'esercizio 29, compilandoli regolarmente con GCC.

Risposte alle domande:

1. Che parametri devo passare in linea di comando?

Guardando il codice del `main()` nel client, notiamo che l'eseguibile si aspetta che gli vengano passati 3 argomenti dopo il suo nome (`argc < 4`).

I parametri da passare sono, in ordine: il nome della funzione da chiamare e i due operandi.

Esempio di utilizzo corretto: `./clientREST-GET calcola-somma 10.5 4.5`

2. Cosa si può vedere analizzando lo scambio di dati tramite Wireshark?

Si osserva perfettamente il funzionamento di un'API REST:

- Il client effettua una richiesta HTTP GET formattando l'URL con i parametri "serializzati": `GET /calcola-somma?param1=10.500000¶m2=4.500000 HTTP/1.1`
- Il server risponde con un classico pacchetto HTTP 200 OK, ma invece di inviare codice HTML, nel Payload (Body) restituisce i dati strutturati formattati in JSON:

```
{  
  "somma": 15.000000  
}
```

Questo dimostra come i Webservice REST usino il protocollo web per lo scambio di dati "raw" tra macchine e non per le interfacce umane.

3. Qual è la signature della funzione `calcolaSomma()` sul server e sul client? Perché ha senso che siano uguali?

La signature (firma della funzione) è identica in entrambi i file: `float calcolaSomma(float val1, float val2)`.

Questo ha un grandissimo senso architetturale ed è alla base del concetto di **RPC** (**Remote Procedure Call**):

- **Sul Server:** la funzione contiene la vera *Business Logic*, ovvero svolge effettivamente il calcolo matematico `val1 + val2` e restituisce il risultato.
- **Sul Client:** la funzione funge da **Stub** (un intermediario/simulacro). Il suo scopo è "nascondere" al programmatore principale tutta la complessità della rete. Lo sviluppatore che scrive il `main()` del client può limitarsi a invocare `calcolaSomma(10, 5)` credendo di eseguire una normale funzione C locale. In realtà, lo Stub intercetta la chiamata, *serializza* i parametri nell'URL, effettua la richiesta di rete HTTP, riceve la risposta, *deserializza* il JSON per estrarne il numero puro, e lo restituisce al `main()`.

Comandi utili:

```
# Compila server e client
gcc serverHTTP-REST.c network.c -o serverHTTP-REST
gcc clientREST-GET.c network.c -o clientREST-GET

# Avvia il server su un terminale
./serverHTTP-REST

# Esegui il client su un altro terminale
./clientREST-GET calcola-somma 10.5 4.5
```

30 Client Java

DOMANDA

Esercizio 2

- Prendere in considerazione il file `ClientREST.java`
 - Dopo aver installato l'ambiente base di Java (già presente in Lab Delta) si può compilare con `javac ClientREST.java`
 - Ed eseguire con `java ClientREST`
- Il fatto che il server sia fatto in C e il client in Java è un problema o un'opportunità? Perché?

RISPOSTA

Soluzione Esercizio 30: Webservice REST Client in Java

Operazioni Preliminari

Ho creato la cartella `es30 - Client Java`, copiato i file necessari e avviato il server `serverHTTP-REST` (scritto in C).

Successivamente, ho compilato il client in Java (`ClientREST.java`) e l'ho eseguito per effettuare la chiamata REST.

Risposte alle domande:

1. **Il fatto che il server sia fatto in C e il client in Java è un problema o un'opportunità? Perché?**

È senza dubbio una grandissima **opportunità** e rappresenta uno dei punti di forza fondamentali delle architetture REST e dei Web Service in generale: **l'interoperabilità e il disaccoppiamento**.

Ecco perché:

- **Indipendenza dal Linguaggio e Piattaforma:** Utilizzando protocolli di rete standard (come HTTP su TCP/IP) e formati di scambio dati agnostici (come stringhe di testo o JSON), i due sistemi non hanno bisogno di condividere lo stesso ecosistema. "Parlano" entrambi la lingua universale di HTTP, decodificandola con le rispettive librerie standard.
- **Scelta dello strumento migliore (Best Tool for the Job):** Permette agli sviluppatori di usare il linguaggio più adatto per ogni strato applicativo. Ad esempio, il server in C garantisce performance elevate, controllo a basso livello e basso overhead, mentre il client in Java offre portabilità estrema (Write Once, Run Anywhere) e librerie avanzate per eventuali interfacce grafiche (GUI) o integrazioni enterprise.
- **Decoupling nello Sviluppo:** Team diversi possono sviluppare il backend e il frontend in completa autonomia lavorando con le proprie tecnologie preferite. Finché viene rispettato il "contratto" (l'API REST, ovvero l'endpoint, i parametri e la struttura della risposta), l'uso di linguaggi diversi sotto il cofano è totalmente trasparente all'altra parte.

Al contrario, se si usassero paradigmi RPC strettamente legati a uno specifico linguaggio (come Java RMI per Java), si sarebbe vincolati a impiegare la medesima tecnologia sia per il client che per il server, impedendo questo grado di flessibilità.

Comandi utili:

```
# Compila e avvia il server in C  
gcc serverHTTP-REST.c network.c -o serverHTTP-REST  
./serverHTTP-REST  
  
# In un altro terminale, compila ed esegui il client in Java  
javac ClientREST.java  
java ClientREST calcola-somma 4 5
```

31 Numeri Primi

DOMANDA

Esercizio 3

- Estendere il webservice `serverHTTP-REST.c` in modo che esponga un secondo servizio relativo al calcolo dei numeri primi compresi nell'intervallo $[min, max]$
 - Si tragga spunto dal programma `prime-number-interval.c`
- Estendere il file `ClientREST.java` in modo da poter chiamare, a scelta, entrambe le funzionalità della nuova API
- Provare il client con il calcolo dei numeri primi compresi nell'intervallo $[1, 1000000]$
 - Quanto tempo ci mette?
 - E' stato necessario tradurre l'algoritmo dei numeri primi in Java? Perché?

RISPOSTA

Soluzione Esercizio 31: Estensione del Webservice con Numeri Primi Operazioni Preliminari

1. Ho copiato i file necessari nella cartella `es31 - Numeri Primi`.
2. Ho modificato il file `serverHTTP-REST.c` implementando un meccanismo di routing per intercettare l'URL `/calcola-primi`. Una volta intercettato, il server esegue l'algoritmo per i numeri primi (preso da `prime-number-interval.c`) e invia gradualmente la lista formattata in JSON tramite la socket HTTP.
3. Ho esteso il client in `ClientREST.java` aggiungendo un metodo `calcolaPrimi(int min, int max)` che si occupa di formattare l'URL e leggere le risposte del server (restituendo i dati riga per riga per non saturare la memoria in caso di payload enormi).
4. Ho compilato ed eseguito il server C in background e lanciato il client Java con il comando `time java ClientREST calcola-primi 1 1000000`.

Risposte alle domande:

1. **Provare il client con il calcolo dei numeri primi compresi nell'intervallo $[1, 1000000]$. Quanto tempo ci mette?**

Il tempo totale impiegato dall'esecuzione completa per calcolare un milione di numeri primi (con l'algoritmo non ottimizzato, $O(N^2)$) è stato di circa:

- **Tempo Reale (real):** 2 minuti e 19 secondi

Questo tempo dipende pesantemente dalla CPU della macchina su cui gira il server e dall'inefficienza dell'algoritmo (visto che calcola i moduli fino a $i/2$), ma dimostra chiaramente che il client è in grado di attendere e gestire connessioni prolungate fino a elaborazione conclusa.

2. **E' stato necessario tradurre l'algoritmo dei numeri primi in Java? Perché?**

Assolutamente NO.

Non è stato necessario tradurre una singola riga di logica matematica in Java. L'intero algoritmo di calcolo risiede ed è eseguito sul **server (in C)**.

Il client Java funge esclusivamente da *Stub*: il suo unico compito è codificare i parametri nell'URL della richiesta GET, inoltrarla tramite HTTP e restare in ascolto sulla socket per leggere (deserializzare) il risultato.

Questo mette in evidenza un altro grande vantaggio dell'architettura REST (e delle architetture client-server basate su RPC): la **centralizzazione della Business Logic**. Il carico computazionale gravoso viene demandato a una macchina remota dedicata (il server C), senza che il client debba possedere le capacità o conoscere le regole per risolverlo, ma preoccupandosi solamente della presentazione all'utente.

32 Thread REST

DOMANDA

Esercizio finale

- Prendere in considerazione il file `clientThreadREST.java`
 - Esso invoca `calcolaSomma()` in 3 thread concorrenti
 - Però la macchina su cui gira il server chiamato è la stessa e quindi non ci guadagno in prestazioni
 - Come si potrebbe modificare il codice in modo che le 3 invocazioni finiscano su 3 macchine diverse?
 - Bisogna modificare anche il codice del server?
- Si riconsideri il servizio che calcola i numeri primi nell'intervallo $[min, max]$ costruito nell'esercizio precedente
 - Si trovi un modo efficiente, sfruttando diversi server in rete, per calcolare i numeri primi tra 1 e 1000000
 - Di quanto migliorano le prestazioni?
 - Devo modificare anche il codice del server?

RISPOSTA

Soluzione Esercizio 32: Multithreading e Calcolo Parallelo Distribuito

Risposte alle domande:

1. **Come si potrebbe modificare il codice (del client) in modo che le 3 invocazioni finiscano su 3 macchine diverse?**

Basterebbe passare al costruttore `RESTAPI` (nel `main` di `ClientThreadREST.java`) gli indirizzi IP delle 3 macchine reali invece di passare `127.0.0.1` fisso a tutti e tre. Esempio:

```
RESTAPI service1 = new RESTAPI("192.168.1.10", args[0], args[1], args[2]);
RESTAPI service2 = new RESTAPI("192.168.1.11", args[0], args[1], args[2]);
RESTAPI service3 = new RESTAPI("192.168.1.12", args[0], args[1], args[2]);
```

2. **Bisogna modificare anche il codice del server?**

Se le macchine sono 3 host fisici o virtuali separati in rete, **NO**. Si può semplicemente copiare lo stesso eseguibile `serverHTTP-REST` identico (che ascolta sulla porta 8000) sulle 3 macchine e avviarlo. Avendo IP diversi, non ci saranno conflitti.

(Tuttavia, se volessimo simulare il cluster testandolo in locale sulla nostra singola macchina loopback 127.0.0.1, dovremmo modificare una riga del server per permettergli di ascoltare su porte diverse, es. 8000, 8001, 8002, per evitare il classico errore di rete "Address already in use").

Il server che abbiamo costruito all'esercizio 31 accetta già due parametri arbitrari `param1` e `param2` nell'URL per estrarre `min` e `max`. Basterà unicamente che il client Java formuli gli URL con i parametri opportuni frammentati. La bellezza dei Web Service REST è proprio questa: finché l'interfaccia/endpoint non varia, la scalabilità e le logiche di partizionamento applicativo si possono gestire senza toccare l'implementazione del servizio vero e proprio.

3. **Si trovi un modo efficiente, sfruttando diversi server in rete, per calcolare i numeri primi tra 1 e 1000000.**

Il calcolo dei numeri primi è un problema altamente parallelizzabile (definito in gergo *embarrassingly parallel*). Possiamo sfruttare i 3 thread del client per contattare i 3 server inviando a ciascuno un **sotto-intervallo** diverso da calcolare. Ad esempio, una ripartizione base potrebbe essere:

- **Thread 1 (Server A):** invoca l'intervallo [1, 333333]
- **Thread 2 (Server B):** invoca l'intervallo [333334, 666666]
- **Thread 3 (Server C):** invoca l'intervallo [666667, 1000000]

4. **Di quanto migliorano le prestazioni?**

Essendo il carico diviso su 3 macchine distinte che calcolano in perfetta contemporanea, il tempo si abbasserà drasticamente, ma il guadagno reale dipenderà da come vengono scelti gli intervalli.

L'importanza del bilanciamento del carico (Load Balancing):

Poiché l'algoritmo C esegue un ciclo di divisioni fino a $i/2$, la sua complessità computazionale cresce in modo quadratico rispetto al valore dei numeri processati. Se usassimo la suddivisione lineare di base (333.333 elementi ciascuno), il Thread 3 (che calcola i numeri più alti) impiegherebbe molto più tempo del Thread 1, creando un "collo di bottiglia" che impedisce un miglioramento proporzionale.

Ipotesi di partizionamento alternativo bilanciato (Ottimizzato):

Per ottenere un bilanciamento perfetto in cui il carico (e quindi il tempo d'esecuzione) è identico per tutti e 3 i server, dobbiamo stringere gli intervalli man mano che i numeri si fanno più grandi. Sapendo che le operazioni totali per arrivare a N crescono proporzionalmente a N^2 , per dividere il lavoro ($N=1.000.000$) in 3 porzioni equivalenti (ognuna contenente il 33% del numero totale di divisioni effettuate dalle CPU), si possono calcolare le soglie matematiche con la radice quadrata:

- **Thread 1 (Server A):** invoca l'intervallo [1, 577350] (*gestisce la fetta più ampia di numeri, ma molto veloci da processare*)
- **Thread 2 (Server B):** invoca l'intervallo [577351, 816496] (*fetta intermedia*)
- **Thread 3 (Server C):** invoca l'intervallo [816497, 1000000] (*fetta più ristretta, compensando l'altissimo costo di processamento di numeri vicini al milione*)

Adottando questa ripartizione ponderata dal lato client, le 3 macchine finiranno di elaborare quasi nello stesso identico istante, sfruttando appieno il parallelismo e ottenendo l'ideale riduzione del tempo di elaborazione totale esattamente a 1/3 (il 33%)!

33 WebSocket Wireshark

DOMANDA

Esercizio 0

Effettuare una cattura Wireshark relativa al funzionamento della chat. Occorre attivare la cattura prima di aprire la URL dalla finestra del browser e ascoltare non solo sull'interfaccia di loopback ma anche su quella verso l'esterno (opzione ANY).

RISPOSTA

Soluzione Esercizio 0: Cattura Wireshark del funzionamento della chat

Operazioni Preliminari:

Ho creato la cartella dell'esercizio (**es33 - WebSocket Esercizio 0**) e vi ho copiato dentro tutto il codice sorgente del progetto "11 WebSocket_Chat".

Analisi della cattura Wireshark:

Avviando una cattura di rete sull'interfaccia "ANY" (o di loopback `lo` se testiamo tutto in locale) e filtrando per la porta del nostro server (es. `tcp.port == 4000`), aprendo il browser e mandando un messaggio in chat si assisterà a una precisa sequenza di eventi di rete che dimostra il funzionamento e il passaggio da HTTP a WebSocket.

Ecco cosa si vede passo per passo nei pacchetti:

1. FASE HTTP (Recupero File Statici)

- Inizialmente si vedrà il classico "3-way handshake" del TCP (SYN, SYN-ACK, ACK) per stabilire la connessione con la porta 4000 del server Node.js.
- Il browser invia una HTTP `GET /` per richiedere la pagina principale.
- Il server Express risponde con un HTTP/1.1 200 OK che contiene il codice HTML della pagina.
- Subito dopo, il browser invierà altre richieste HTTP `GET` per scaricare i file statici referenziati nell'HTML (come `/chat.js` e `/styles.css`).

2. FASE DI UPGRADE (Il passaggio a WebSocket)

- Il client (grazie all'istruzione `io.connect()` nel file JavaScript) inizierà una richiesta speciale verso il server. Nello specifico, invierà una richiesta HTTP `GET` contenente degli header molto particolari:

```
Connection: Upgrade
Upgrade: websocket
Sec-WebSocket-Key: [chiave casuale]
```

- Se il server supporta WebSocket (come nel nostro caso grazie a Socket.IO), risponderà al browser con un pacchetto speciale: HTTP/1.1 101 Switching Protocols.
- Negli header della risposta 101, il server confermerà l'avvenuto "cambio di lingua" ri-dichiarando `Upgrade: websocket` e rispondendo alla chiave di sicurezza.

3. FASE WEBSOCKET (Comunicazione Bidirezionale in Real-Time)

- Da questo momento in poi (dopo il pacchetto 101), Wireshark smetterà di classificare la conversazione come "HTTP". I pacchetti successivi sulla stessa connessione TCP saranno etichettati col protocollo "WebSocket".
- Quando l'utente preme il bottone per inviare un messaggio nella chat, Wireshark mostrerà un pacchetto "WebSocket Text Frame".
- Poiché la comunicazione non è cifrata (non stiamo usando WSS ma semplice WS), cliccando sul payload del pacchetto in Wireshark sarà possibile leggere in chiaro il messaggio inviato.
- Essendo mediato dalla libreria Socket.IO, il testo nel payload apparirà con un piccolo prefisso numerico usato dal protocollo interno della libreria, seguito dal nostro oggetto JSON. Ad esempio:
`42["message", {"sender":"Mario", "message":"Ciao a tutti!"}]`
- Subito dopo, si vedranno pacchetti WebSocket simili in direzione opposta (Server -> Client) che rappresentano il "broadcasting" dell'evento `UploadChat` verso gli altri client connessi per aggiornare le loro interfacce.

Conclusione:

La cattura dimostra visivamente il più grande vantaggio dei WebSocket: al contrario del polling HTTP, non c'è bisogno di instaurare continuamente nuove connessioni o inviare pesanti Header HTTP per ogni singolo messaggio. Una volta effettuato l'handshake e l'Upgrade (101 Switching Protocols), il tunnel TCP resta permanentemente aperto e i messaggi viaggiano al suo interno come frame leggerissimi e a bassissima latenza.

34 WebSocket Restyling

DOMANDA

Esercizio 1

Modificare a piacimento il contenuto del file `public/index.html` e valutarne l'impatto grafico.

RISPOSTA

Soluzione Esercizio 1: Modifica grafica e impatto visivo

Operazioni Preliminari:

1. Ho copiato tutto il codice sorgente originale in `es34 - WebSocket Esercizio 1`.
2. Ho stravolto il file `public/index.html` e riscritto da zero il foglio di stile `public/styles.css`.
3. Ho avviato il server Node.js in background. Per visualizzare il risultato, basta collegarsi a `http://localhost:4000`.

Analisi dell'impatto grafico (Il restyling):

L'interfaccia originale era molto "grezza": un contenitore bianco su sfondo bianco, pulsanti squadrati e tipografia di sistema, caratteristiche tipiche di un prototipo base. Per modernizzare l'interfaccia e offrire un impatto visivo ("WOW factor") all'altezza delle applicazioni web contemporanee, ho implementato le seguenti migliorie strutturali e stilistiche in CSS puro:

1. Il Tema (Dark Gradient):

Ho sostituito lo sfondo bianco con un gradiente diagonale scuro (da blu notte profondo a indaco). Questo fa risaltare il riquadro centrale della chat e riduce l'affaticamento visivo (Dark Mode nativa).

2. Glassmorphism (Effetto Vetro Smerigliato):

Invece di un semplice colore a tinta unita per il container della chat, ho utilizzato uno sfondo semi-trasparente (`rgba(255, 255, 255, 0.05)`) combinato con la direttiva `backdrop-filter: blur(12px)`. Questo simula l'effetto di un pannello di vetro sospeso sopra lo sfondo, conferendo una profondità tridimensionale tipica delle UI di iOS o di Windows 11.

3. Tipografia Moderna:

Ho rimosso il font di default (Nunito/System) e importato *Poppins* da Google Fonts. Le sue geometrie rotonde e i pesi marcati (`font-weight: 600`) donano alla WebApp un tono molto più amichevole, netto e leggibile.

4. Animazioni e Dettagli (Micro-Interactions):

- Ho trasformato il bottone "Send" da un blocco rettangolare a un pulsante circolare flottante inserendo un'icona SVG vettoriale minimalista (un aeroplanino di carta).
- Ho aggiunto degli effetti di *hover* (il bottone si ingrandisce al passaggio del mouse e si rimpicciolisce al click, animato fluidamente tramite `transition: all 0.3s ease`).

- I nuovi messaggi che appaiono in chat, invece di spuntare di scatto, hanno ora un'animazione **fadeIn** CSS (**@keyframes**) che li fa "scivolare" dolcemente dal basso verso l'alto rendendo l'esperienza visiva fluida e gratificante.

Il risultato è una chat che, pur mantenendo l'identica infrastruttura e tecnologia Web-Socket dell'esercizio precedente (Node.js e Socket.io), appare all'utente finale come un'applicazione premium, dimostrando quanto l'impatto grafico (Frontend) possa alterare drasticamente la percezione di qualità del software, a parità di backend.

35 WebSocket porta 80

DOMANDA

Esercizio 2

Modificare il sorgente del codice in modo da far ascoltare il server sulla porta 80 invece che 4000. Quali altri accorgimenti sono necessari per farlo funzionare: lato server? Lato client?

RISPOSTA

Esercizio 35 - WebSocket porta 80

Lato Server

In sistemi Unix-like (come Linux e macOS), le porte al di sotto della 1024 sono considerate porte "privilegiate" o "well-known". Per far sì che un programma possa mettersi in ascolto sulla porta 80, sono necessari permessi di amministratore. Di conseguenza, il comando per avviare il server non sarà più solamente "node server.js", ma dovrà essere eseguito con i privilegi di root, ad esempio:

```
sudo node server.js
```

(In alternativa, si possono concedere all'eseguibile di node le capabilities per fare bind su porte privilegiate usando: `setcap cap_net_bind_service=+ep /path/to/node`).

Lato Client

Dal punto di vista dell'utente finale, per connettersi alla chat non sarà più necessario specificare il numero di porta nell'URL del browser (la porta 80 è quella predefinita per il protocollo HTTP). L'indirizzo da visitare sarà semplicemente "http://localhost/" (oppure l'indirizzo IP del server).

Per quanto riguarda il codice JavaScript del client (`public/chat.js`), la chiamata per instaurare la connessione WebSocket è attualmente:

```
var websocket = io.connect();
```

Poiché questa chiamata non specifica esplicitamente un host e una porta, la libreria `socket.io` utilizzerà automaticamente lo stesso host e la stessa porta da cui è stata scaricata la pagina HTML (ovvero la porta 80). Pertanto, non è necessario apportare alcuna modifica al codice lato client per farlo funzionare. Se invece il codice avesse utilizzato un URL hard-coded come `io.connect('http://localhost:4000')`, sarebbe stato necessario modificarlo o rimuovere l'URL.

36 WebSocket rinomina eventi

DOMANDA

Esercizio 3

Modificare il sorgente del codice in modo da cambiare nome ai seguenti eventi:

- `message` → `messaggio`
- `UploadChat` → `aggiornamento`

RISPOSTA

Esercizio 36 - Rinomina Eventi WebSocket

Modifiche effettuate al codice sorgente:

1. Lato Server (`server.js`):

- Modificato l'evento in ascolto da `'message'` a `'messaggio'`:
`websocket.on('messaggio', function(data) { ... });`
- Modificato l'evento emesso in broadcast da `'UploadChat'` a `'aggiornamento'`:
`io.sockets.emit('aggiornamento', data);`

2. Lato Client (`public/chat.js`):

- Modificato il nome dell'evento inviato al click del pulsante da `'message'` a `'messaggio'`:
`websocket.emit('messaggio', { ... });`
- Modificato il nome dell'evento in ascolto per ricevere i messaggi da `'UploadChat'` a `'aggiornamento'`:
`websocket.on('aggiornamento', function(data) { ... });`

37 WebSocket test dispositivi

DOMANDA

Esercizio 4

Se si dispone di due PC in grado di dialogare sulla stessa rete IP (oppure un PC per il server e uno smartphone per il client sempre in grado di dialogare tra loro a livello IP) provare ad accedere al server che è su Linux mediante diversi tipi di browser e di sistemi operativi. Cosa si può notare?

RISPOSTA

Esercizio 37 - Test WebSocket da diversi dispositivi

Effettuando il test di accesso da vari dispositivi (come PC Windows, macOS, smartphone Android o iOS) e browser differenti (Chrome, Firefox, Safari, Edge, ecc.), si possono notare due aspetti principali:

1. **Interoperabilità e Standardizzazione:** L'applicazione funziona in modo identico e fluido su tutti i dispositivi e browser moderni senza necessità di installare software aggiuntivo o applicazioni dedicate sul client. Questo perché le WebSocket sono uno standard web nativamente supportato da tutti i principali browser e operano al di sopra del protocollo HTTP/TCP.
2. **Sincronizzazione in tempo reale (Real-time):** Qualsiasi messaggio inviato da uno dei dispositivi (ad esempio dallo smartphone) compare istantaneamente sugli schermi di tutti gli altri dispositivi collegati (ad esempio i PC), confermando che il server riesce a mantenere aperte le connessioni persistenti (full-duplex) con tutti i client simultaneamente e a fare broadcast degli eventi in tempo reale, indipendentemente dal sistema operativo o dal browser utilizzato.

Nota operativa:

Perché l'accesso abbia successo, è fondamentale che i client nella rete locale conoscano l'indirizzo IP del server (es. 192.168.x.x) e che l'eventuale firewall sul server Linux (es. `ufw` o `iptables`) sia configurato per accettare connessioni in entrata sulla porta utilizzata dal server Node.js (es. porta 4000). Altrimenti la pagina non si caricherà.

38 WebSocket typing

DOMANDA

Esercizio 5

Modificare il sorgente del codice per fare in modo che ad ogni utente connesso alla chat arrivi nella console il messaggio "l'utente sta scrivendo..." .

NOTA: Lato client, bisogna spedire al server un evento apposito (ad es. "typing") quando l'utente scrive sulla tastiera (catturando l'evento di sistema "keydown"). Lato server, la chiamata `websocket.broadcast.emit('typing', data)` rilancia l'evento "typing" a tutti i client connessi tranne che a quello dalla quale si è ricevuto il messaggio. Lato client occorre infine gestire la ricezione del messaggio "typing" che arriva dal server (vedere gestione del messaggio "UploadChat").

RISPOSTA

Esercizio 38 - Segnalazione "l'utente sta scrivendo..."

Modifiche effettuate al codice sorgente:

1. Lato Client (`public/chat.js`):

- È stato aggiunto un listener (`'keydown'`) sull'input testuale per intercettare la pressione dei tasti. Quando l'utente digita, viene emesso un evento `'typing'` inviando al server il nome del mittente:

```
message.addEventListener('keydown', function(){
    websocket.emit('typing', sender.value);
});
```

- È stato aggiunto un listener per gestire l'evento `'typing'` in arrivo dal server. Per evitare che il messaggio sovrasciva eventuali altri utenti che stanno digitando contemporaneamente, viene verificato se la stringa è già presente nel div "feedback" e, se non lo è, viene aggiunta (accodata) e viene scritto un log nella console:

```
websocket.on('typing', function(data){
    if (feedback.innerHTML.indexOf(data + '␣sta␣scrivendo') === -1) {
        feedback.innerHTML += '<p><em>' + data + '␣sta␣scrivendo...</em></p>';
        console.log(data + "␣sta␣scrivendo...");
    }
});
```

(Nota: L'evento `'UploadChat'` preesistente pulisce già il div "feedback" azzerandone il contenuto quando un messaggio viene effettivamente inviato, rimuovendo quindi la scritta "sta scrivendo...").

2. Lato Server (`server.js`):

- È stata aggiunta la gestione dell'evento `'typing'`. Quando il server lo riceve, usa la funzione "broadcast" per re-inviare lo stesso evento a tutti gli altri client connessi, escludendo il client mittente:

```
websocket.on('typing', function(data){
    websocket.broadcast.emit('typing', data);
});
```

39 MQTT retain

DOMANDA

- Partendo dall'esercizio mostrato nelle istruzioni, cosa succede se pubblico una temperatura prima di aver lanciato il subscriber?
 - Provare il publisher con l'opzione **-retain**

RISPOSTA

Soluzione esercizi MQTT - Pub-sub1 e Pub-sub2

Partendo dalle istruzioni mostrate per pubblicare e iscriversi a un topic:

1. Cosa succede se pubblico una temperatura prima di aver lanciato il subscriber?

Nel funzionamento standard di MQTT (senza l'opzione **-retain**), se un publisher invia un messaggio su un topic quando non ci sono subscriber attualmente connessi e in ascolto per quel topic, il messaggio viene perso. Il broker non memorizza i messaggi passati, si limita a smistarli ai client online in quel preciso istante. Lanciando il subscriber in un momento successivo, quest'ultimo rimarrà semplicemente in attesa dei nuovi messaggi futuri.

2. Provare il publisher con l'opzione **-retain** (e cosa comporta)

Se il comando del publisher viene lanciato aggiungendo l'opzione **-retain** (oppure **-r**), ad esempio:

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \  
-p 8883 -u univr-studenti -P MQTT-esercitazione2026 \  
-t temperatura -m "24" --retain
```

Il broker MQTT memorizzerà (tratterrà) quest'ultimo messaggio per il topic specificato. Di conseguenza, se avvia un subscriber successivamente, non appena si iscriverà al topic riceverà immediatamente l'ultimo messaggio memorizzato (il "24"), senza dover attendere una nuova pubblicazione. (Nota: il broker trattiene sempre e solo l'ultimo messaggio per cui è stato specificato il flag **retain**).

40 MQTT due sensori

DOMANDA

- Partendo dall'esercizio mostrato nelle istruzioni creare un'applicazione pub/sub con 2 sensori di temperatura relativi a 2 stanze diverse. Quante finestre di terminale devo aprire?

RISPOSTA

Soluzione esercizi MQTT - Due sensori

Domanda: Partendo dall'esercizio mostrato nelle istruzioni creare un'applicazione pub/sub con 2 sensori di temperatura relativi a 2 stanze diverse. Quante finestre di terminale devo aprire?

Risposta:

Devi aprire un totale di **3** finestre di terminale:

- **1 finestra** per il Subscriber (che si metterà in ascolto dei dati provenienti da entrambe le stanze).
- **1 finestra** per il Publisher del Sensore 1 (Stanza 1).
- **1 finestra** per il Publisher del Sensore 2 (Stanza 2).

(**Nota:** non serve una finestra per il broker poiché stiamo usando quello remoto in cloud su HiveMQ).

Ecco i comandi da inserire nelle 3 finestre:

— **Finestra 1: SUBSCRIBER** —

Per metterti in ascolto su entrambe le stanze, puoi usare il carattere jolly "#" (che include tutti i sotto-topic di "temperatura"), e l'opzione "-v" (verbose) per stampare a schermo anche il nome del topic oltre al valore.

```
mosquitto_sub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
-p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
-t "temperatura/#" -v
```

— **Finestra 2: PUBLISHER (Sensore Stanza 1)** —

Pubblica un valore di temperatura (es. 22) sul topic specifico della stanza 1.

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
-p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
-t "temperatura/stanza1" -m "22"
```

— **Finestra 3: PUBLISHER (Sensore Stanza 2)** —

Pubblica un valore di temperatura (es. 24) sul topic specifico della stanza 2.

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
-p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
-t "temperatura/stanza2" -m "24"
```

Domanda: Partendo dall'esercizio precedente, come fare per avere un unico subscriber per entrambe le temperature? Come si fa a distinguere da quale stanza proviene la temperatura?

Risposta:

1. **Come avere un unico subscriber per entrambe le temperature:**
Per ricevere i messaggi da più topic simultaneamente utilizzando un solo subscriber,

si utilizzano i "wildcard" (caratteri jolly) nei topic. Se i publisher pubblicano sui topic `temperatura/stanza1` e `temperatura/stanza2`, il subscriber può iscriversi al topic `temperatura/#` (wildcard multi-livello) oppure `temperatura/+` (wildcard a singolo livello). In questo modo, intercetterà tutti i messaggi pubblicati in qualsiasi sotto-categoria di "temperatura". (Come mostrato già nel comando del subscriber nella Parte 1).

2. Come distinguere da quale stanza proviene la temperatura:

Di default, `mosquitto_sub` stampa a schermo solamente il "payload" del messaggio (il valore numerico, ad es. "22" o "24"), rendendo impossibile capire da quale stanza provenga. Per ovviare a questo problema, bisogna lanciare il comando `mosquitto_sub` aggiungendo l'opzione `-v` (che sta per "verbose"). In modalità verbose, il client stampa sia il topic completo sia il messaggio.

Output di esempio che vedrai sul terminale:

```
temperatura/stanza1 22
temperatura/stanza2 24
```

41 MQTT temp e umidita

DOMANDA

- Prova a pubblicare un valore di umidità relativa (topic "UR"); il subscriber interessato alle temperature lo riceve? Come si fa a creare un subscriber interessato all'umidità? Costruire un'applicazione pub/sub con 4 finestre per produrre e visualizzare sia valori di temperatura sia valori di umidità.

RISPOSTA

Soluzione esercizio MQTT - es41

Domanda 1: Prova a pubblicare un valore di umidità relativa (topic "UR"); il subscriber interessato alle temperature lo riceve?

Risposta 1: No, il subscriber non lo riceve. Il protocollo MQTT si basa su un filtraggio rigoroso tramite i topic. Un subscriber iscritto al topic "temperatura" (o "temperatura/#") ignorerà completamente tutti i messaggi pubblicati su topic differenti, come "UR".

Domanda 2: Come si fa a creare un subscriber interessato all'umidità?

Risposta 2: Bisogna lanciare un nuovo client `mosquitto_sub` specificando come argomento per il parametro `-t` il topic dell'umidità, ovvero "UR".

Domanda 3: Costruire un'applicazione pub/sub con 4 finestre per produrre e visualizzare sia valori di temperatura sia valori di umidità.

Risposta 3: Per avere un sistema completo servono 4 terminali separati. Di seguito i comandi da inserire in ciascuna finestra (utilizzando sempre il broker cloud di HiveMQ):

— Finestra 1: SUBSCRIBER TEMPERATURA —

```
mosquitto_sub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
-p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
-t "temperatura/#" -v
```

— Finestra 2: SUBSCRIBER UMIDITA' —

```
mosquitto_sub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
-p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
-t "UR/#" -v
```

— Finestra 3: PUBLISHER TEMPERATURA —

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
-p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
-t "temperatura/stanza1" -m "22"
```

— Finestra 4: PUBLISHER UMIDITA' —

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
-p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
-t "UR/stanza1" -m "60"
```

42 MQTT wireshark

DOMANDA

- Aprire e studiare con Wireshark il file PCAP contenuto nello ZIP
 - Quale protocollo di livello trasporto utilizza MQTT? Quali sono le porte?
 - Trovare le fasi di publish e subscribe. Quante connessioni TCP apre un subscriber? Quante connessioni TCP apre un publisher?

RISPOSTA

Soluzione esercizio MQTT - es42 (Analisi PCAP)

Domanda 1: Quale protocollo di livello trasporto utilizza MQTT? Quali sono le porte?

Risposta 1:

MQTT utilizza il protocollo di livello trasporto TCP.

La porta standard utilizzata (come si vede dal file PCAP) è la 1883 per le connessioni in chiaro (non cifrate). Qualora si utilizzasse MQTT over TLS (comunicazione cifrata), la porta standard sarebbe la 8883.

Domanda 2: Trovare le fasi di publish e subscribe. Quante connessioni TCP apre un subscriber? Quante connessioni TCP apre un publisher?

Risposta 2:

Entrambi i client (publisher e subscriber) instaurano esattamente 1 sola connessione TCP con il broker per svolgere il proprio lavoro.

- **Il Subscriber apre 1 singola connessione TCP persistente:** esegue il 3-way handshake, invia il messaggio MQTT di "Connect Command", riceve il "Connect Ack", dopodiché invia la "Subscribe Request" e mantiene aperta quella stessa connessione TCP per rimanere in ascolto dei futuri messaggi.
- **Il Publisher apre anch'esso 1 singola connessione TCP:** esegue l'handshake, si connette ("Connect Command"), invia il dato ("Publish Message") e al termine invia una richiesta di disconnessione ("Disconnect Req"), chiudendo subito dopo la singola connessione TCP utilizzata.

Domanda 3: Documentare i comandi utilizzati per aprire e analizzare sia da interfaccia grafica che utilizzando la CLI.

Risposta 3:

— **Analisi da Interfaccia Grafica (Wireshark)** —

Puoi aprire il file lanciando da terminale:

```
wireshark mqttt-localhost.pcapng
```

Una volta aperta l'interfaccia di Wireshark, per isolare immediatamente e studiare le fasi esatte di publish e subscribe ignorando il rumore di fondo, è sufficiente scrivere nella barra dei filtri in alto (Display Filter) la parola chiave: `mqttt` (e poi premere Invio).

— **Analisi da riga di comando (CLI)** —

Se vuoi analizzare il PCAP interamente da riga di comando usando TShark (la versione CLI di Wireshark), il comando per filtrare solo i pacchetti MQTT è:

```
tshark -r mqttt-localhost.pcapng -Y "mqttt"
```

In alternativa, se tshark non è disponibile, puoi usare il classico `tcpdump` per visualizzare tutti i pacchetti TCP scambiati sulla porta 1883:


```
tcpdump -r mqttt-localhost.pcapng -n port 1883
```

43 MQTT messaggistica univr

DOMANDA

Si vuole costruire con MQTT un servizio di messaggistica universitaria:

- Il rettore può leggere tutti i messaggi
- La segreteria può leggere i messaggi dai docenti e dagli studenti
- I docenti possono leggere i messaggi dai docenti e dagli studenti
- Gli studenti possono leggere solo i messaggi degli altri studenti

RISPOSTA

Soluzione esercizio MQTT - es43

Obiettivo: Costruire un servizio di messaggistica universitaria con MQTT rispettando specifici permessi di lettura per i vari ruoli.

Per risolvere questo problema, la soluzione ottimale consiste nel definire una gerarchia di topic basata sul mittente (chi genera il messaggio). Possiamo definire la seguente struttura di topic radice:

`messaggi/<ruolo_mittente>`

I possibili topic in cui i vari utenti andranno a pubblicare saranno quindi:

- `messaggi/rettore`
- `messaggi/segreteria`
- `messaggi/docenti`
- `messaggi/studenti`

In base alle regole imposte dall'esercizio, ecco come si dovranno configurare le iscrizioni (subscribers) per ciascun ruolo:

1. IL RETTORE

Regola: Può leggere tutti i messaggi.

Soluzione: Il rettore si iscriverà utilizzando il carattere jolly "#" per intercettare qualsiasi messaggio sotto la radice "messaggi/".

Topic di sottoscrizione: `messaggi/#`

2. LA SEGRETERIA

Regola: Può leggere i messaggi dai docenti e dagli studenti.

Soluzione: La segreteria dovrà iscriversi a due topic separati (si può fare lanciando il client con due flag `-t`).

Topic di sottoscrizione: `messaggi/docenti` e `messaggi/studenti`

3. I DOCENTI

Regola: Possono leggere i messaggi dai docenti e dagli studenti.

Soluzione: Esattamente come la segreteria, i docenti si iscriveranno a questi due topic.

Topic di sottoscrizione: `messaggi/docenti` e `messaggi/studenti`

4. GLI STUDENTI

Regola: Possono leggere solo i messaggi degli altri studenti.

Soluzione: Gli studenti si iscriveranno in ascolto unicamente sul topic dedicato a loro.

Topic di sottoscrizione: messaggi/studenti

COMANDI PRATICI DA ESEGUIRE SUL TERMINALE

(I comandi utilizzano il broker remoto HiveMQ indicato negli esercizi precedenti)

»> **COMANDI PER LEGGERE (SUBSCRIBER)** «<

Rettore (ascolta tutto):

```
mosquitto_sub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
               -p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
               -t "messaggi/#" -v
```

Segreteria e Docenti (ascoltano docenti e studenti):

```
mosquitto_sub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
               -p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
               -t "messaggi/docenti" -t "messaggi/studenti" -v
```

Studenti (ascoltano solo studenti):

```
mosquitto_sub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
               -p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
               -t "messaggi/studenti" -v
```

»> **COMANDI PER SCRIVERE (PUBLISHER)** «<

Pubblicare come Rettore:

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
               -p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
               -t "messaggi/rettore" -m "Avviso_dal_Rettore"
```

Pubblicare come Segreteria:

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
               -p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
               -t "messaggi/segreteria" -m "Avviso_dalla_Segreteria"
```

Pubblicare come Docenti:

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
               -p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
               -t "messaggi/docenti" -m "Lezione annullata"
```

Pubblicare come Studenti:

```
mosquitto_pub -h e0d996a0720a4a25ae1a34becc9e8a90.s1.eu.hivemq.cloud \
               -p 8883 -u univr-studenti -P MQTT-esercitazione2026 \
               -t "messaggi/studenti" -m "Cerco appunti"
```